

Herança em Java

Você vai entender a herança de classes em Java, fundamental na programação orientada a objetos e amplamente usada nas bibliotecas, incluindo o framework Collections. Verá a hierarquia de herança em Java e o princípio da substituição de Liskov. Você também vai explorar o framework Collections com exemplos práticos e, finalmente, conhecerá os principais métodos da classe Object, base de todas as classes em Java. Com muitos exemplos, este conteúdo pretende incentivar a experimentação prática com Java através de compilador dedicado.

Prof. Marlos de Mendonça Corrêa

Preparação

Tenha à sua disposição um computador com kit de desenvolvimento Java (JDK). Dois JDK populares são o Java SE Development Kit (www.oracle.com/java/technologies/downloads) e o OpenJDK (openjdk.org). Acesso à internet também será de grande ajuda para consultar a documentação oficial da linguagem ou tirar dúvidas. Uma vez baixados e instalados, é recomendável instalar uma IDE. Embora não seja imprescindível, uma IDE facilita bastante o desenvolvimento e é uma ferramenta muito útil no processo de aprendizagem. Entre as diversas IDE existentes, o Netbeans (netbeans.apache.org/download/index.html) e o Eclipse (www.eclipse.org/downloads) são algumas das mais utilizadas.

Objetivos

- Descrever a hierarquia de herança em Java.
- Aplicar a herança nas bibliotecas da linguagem Java.
- Empregar os principais métodos de objetos em Java.

Introdução

A orientação a objetos (OO) é um paradigma consagrado no desenvolvimento de software e empregado independentemente da metodologia de projeto – ágil ou prescritiva. A Java é uma linguagem puramente OO, que está entre as mais utilizadas no mundo (Vanshika2024, s. d.). Assim, esse conhecimento é imprescindível para qualquer profissional de TI que queira trabalhar com desenvolvimento de software.

Quando dizemos que a orientação a objetos está no cerne da linguagem Java, isso é mais profundo do que estabelecer que a linguagem suporta esse paradigma. A própria linguagem foi estruturada segundo o paradigma OO. Java está ligada de forma indissociável à orientação a objetos; logo, para extrair todos os recursos que Java tem a oferecer, torna-se fundamental conhecer bem OO.

Na verdade, muito do poder da Java vem das capacidades que a OO trouxe para o desenvolvimento de software. Exemplos disso são a herança e o polimorfismo. Tais conceitos, próprios da OO e não de uma linguagem específica, foram empregados na concepção da linguagem Java, sendo responsáveis por muitas das funcionalidades oferecidas. Em Java, todas as classes descendem direta ou indiretamente da classe Objects.

Por esses motivos, vamos nos aprofundar no conhecimento de herança na linguagem Java. Veremos como a linguagem emprega um conceito chamado interfaces e estudaremos métodos de especial interesse para a manipulação de objetos. Esse processo será acompanhado de atividades para reforçar o aprendizado e fornecer feedback. Ao fim, teremos compreendido uma parte do desenvolvimento em Java que é indispensável para qualquer estudo mais avançado.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Herança e instanciação de objetos em Java

A herança não é uma criação do paradigma de programação orientada a objetos (POO). Você deve estar familiarizado com o termo, que é muito usado para representar o ato de alguém deixar algo para outrem. Em OO, herança possui o mesmo entendimento: é a transmissão de características aos descendentes, é a capacidade de uma entidade legar a outra seus métodos e atributos. Por legar, devemos entender que os métodos e atributos estarão presentes na classe derivada. Para compreendermos melhor, vamos entender a criação de um objeto em Java.

Assista ao vídeo para compreender mais a fundo o conceito de herança e como ele pode auxiliar o desenvolvedor no manejo da complexidade. Além disso, você aprenderá como criar instâncias de objetos em Java.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Hierarquia de classe e herança

Quando definimos uma classe, definimos um mecanismo de criação de objetos. Uma classe define um tipo de dado e cada objeto instanciado pertence ao conjunto formado pelos objetos daquela classe. Nesse conjunto, todos os objetos são do tipo da classe que lhes deu origem.

Como sabemos, uma classe possui métodos e atributos. Os métodos modelam o comportamento do objeto e atuam sobre o seu estado, que é definido pelos atributos. Quando um objeto é instanciado, uma área de memória é reservada para acomodar os métodos e atributos que o objeto deve possuir. Todos os objetos do mesmo tipo terão os mesmos métodos e atributos, porém cada objeto terá a sua própria cópia destes. Considere, a título de ilustração, a definição de classe mostrada no Código 1.

Código 1: Declaração de classe:

```
java

//Pacotes
package com.mycompany.GereEscola;

//Classe
public class Pessoa {
    //Atributos
    protected String nome , nacionalidade , naturalidade;

    //Métodos
    public Pessoa ( String nome , String nacionalidade , String naturalidade ) {
        this.nome = nome;
        this.nacionalidade = nacionalidade;
        this.naturalidade = naturalidade;
    }
    protected void atualizarNome ( String nome ) {
        this.nome = nome;
    }
    protected String recuperarNome ( ) {
        return this.nome;
    }
    protected String recuperarNacionalidade ( ) {
        return this.nacionalidade;
    }
    protected String recuperarNaturalidade ( ) {
        return this.naturalidade;
    }
}
```

Qualquer objeto instanciado a partir dessa classe terá os atributos “nome”, “nacionalidade” e “naturalidade”, além de uma cópia dos métodos mostrados. Na verdade, como String é um objeto de tamanho desconhecido em tempo de programação, as variáveis são, na verdade, referências que irão guardar a localização em memória dos objetos do tipo String, quando esses forem criados.



Resumindo

Quando objetos se relacionam por associação, por exemplo, essa relação se dá de maneira horizontal. Normalmente são relações do tipo “contém” ou “possui”. A herança introduz um novo tipo de relacionamento, inserindo a ideia de “é tipo” ou “é um”. Esse relacionamento vertical origina o que chamamos hierarquia de classes. Uma hierarquia de classe é um conjunto de classes que guardam entre si uma relação de herança (verticalizada), na qual as classes acima generalizam as classes abaixo ou, por simetria, as classes abaixo especializam as classes acima.

Vamos analisar a classe derivada mostrada no Código 2.

Código 2: Declaração de subclasse:

```
java

//Pacotes
package com.mycompany.GereEscola;

//Classe
public class Aluno extends Pessoa {

   public Aluno ( String nome , String nacionalidade , String naturalidade ) {
     super ( nome , nacionalidade , String );
   }
}
```

A classe Aluno estende a classe Pessoa, ou seja, é uma subclasse de Pessoa, formando assim uma relação hierárquica (Aluno deriva de Pessoa). Podemos aplicar a ideia introduzida com o conceito de herança para deixar mais claro o que estamos falando: Aluno é **um tipo de/é uma** Pessoa. Um objeto do tipo Aluno também é do tipo Pessoa. Nesse sentido, como vimos, a classe Pessoa lega seus métodos e atributos à classe Aluno.

Verificando o Código 2, notamos que a classe Aluno não possui métodos e atributos definidos em sua declaração, exceto pelo seu construtor. Logo, ela terá apenas os componentes legados por Pessoa. A instanciação de um objeto do tipo Aluno, nesse caso, levará à reserva de um espaço de memória para guardar apenas os atributos nome, nacionalidade e naturalidade e os métodos de Pessoa e o construtor de Aluno. No nosso exemplo, o objeto seria virtualmente igual a um objeto do tipo Pessoa.



Comentário

O exemplo apresentado tem apenas fim didático e busca mostrar na prática, ainda que superficialmente, o mecanismo de herança. Como os tipos do exemplo são praticamente idênticos, nenhuma utilidade prática se obtém — Aluno não especializa Pessoa. Mas aí também atingimos outro fim: mostramos que a versatilidade da herança está na possibilidade de realizarmos especializações e generalizações no modelo a ser implementado. A especialização ocorre quando a classe derivada particulariza comportamentos, casos em que as subclasses alteram métodos da superclasse.

No Código 3 (Exemplo de especialização), a classe Aluno definida possui um novo atributo — “matricula” — que será gerado automaticamente quando o objeto for instanciado. Embora esse objeto possua os métodos e atributos de Pessoa, ele agora tem um comportamento particular: na instanciação, além dos atributos definidos em Pessoa, ocorre também a definição de “matricula”.

Código 3: Exemplo de especialização:

```
java

//Pacotes
package com.mycompany.GereEscola;

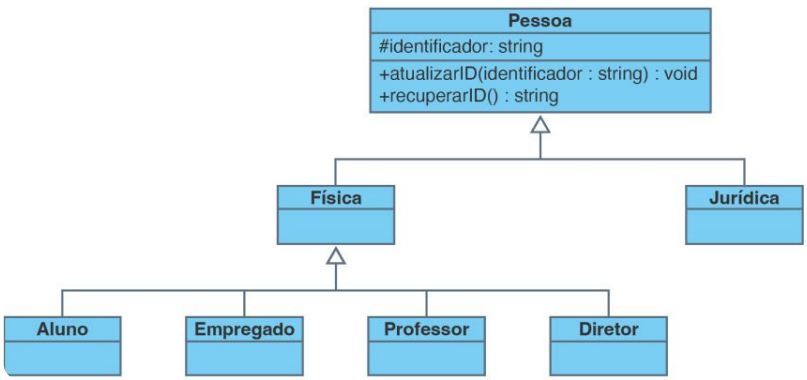
//Importações
import java.util.UUID;

//Classe
public class Aluno extends Pessoa {

//Atributos
    private String matricula;

//Métodos
    public Aluno ( String nome , String nacionalidade , String naturalidade ) {
        super ( nome , nacionalidade , naturalidade );
        matricula = UUID.randomUUID( ).toString( );
    }
}
```

Vejamos a hierarquia de classes mostrada na próxima imagem. Já sabemos que o atributo “identificador” e os métodos que operam sobre ele (“atualizarID” e “recuperarID”) são herdados pelas classes filhas. Também sabemos que um objeto da classe Fisica é também é do tipo Pessoa. Mas vale destacar que a relação de herança se propaga indefinidamente, isto é, um objeto da classe Aluno também é um objeto das classes Fisica e Pessoa.



Especialização de comportamento

A classe Pessoa implementa o comportamento mais genérico de todos. Ela possui um atributo de identificação definido como do tipo “String” (aceita letras, símbolos e números) e métodos que operam sobre esse atributo, como você pode ver no Código 4. As classes Fisica, Juridica e Aluno especializam o método que define o atributo “identificador”, particularizando-o para as suas necessidades específicas, como observamos nos Código 5, Código 6 e Código 7, que mostram as respectivas implementações.

Vejamos o código parcial dos métodos genéricos da classe Pessoa e método "atualizarID" da classe Fisica.

Código 4: Código parcial da classe Pessoa — métodos genéricos:

```
java

//Classe
public class Pessoa {
    ...
    private String identificador;
    ...
    protected void atualizarID ( String identificador ) {
        this.identificador = identificador;
    }
    protected String recuperarID ( ) {
        return this.identificador;
    }
    ...
}
```

Código 5: Método "atualizarID" da classe Fisica:

```
java

protected void atualizarID ( String CPF ) {
    if ( validaCPF ( CPF ) )
        this.identificador = CPF;
    else
        System.out.println ( "ERRO: CPF invalido!" );
}
```

Agora, vejamos o código parcial dos métodos "atualizarID" nas classes Juridica e Aluno.

Código 6: Método "atualizarID" da classe Juridica:

```
java

protected void atualizarID ( String CNPJ ) {
    if ( validaCNPJ ( CNPJ ) )
        this.identificador = CNPJ;
    else
        System.out.println ( "ERRO: CNPJ invalido!" );
}
```

Código 7: Método "atualizarID" da classe Aluno:

```
java

public void atualizarID ( ) {
    if ( this.identificador.isBlank() )
        this.identificador = UUID.randomUUID( ).toString( );
    else
        System.out.println ( "ERRO: Codigo matricula ja existente!" );
}
```

Repare que cada código possui um comportamento específico. Apesar de não exibirmos a implementação do método “validaCPF” e “validaCNPJ”, fica claro que os comportamentos são distintos entre si e entre as demais implementações de “atualizarID”, pois o Cadastro de Pessoas Físicas (CPF) e o Cadastro Nacional de Pessoas Jurídicas (CNPJ) possuem regras de formação diferentes.

Atividade 1

Herança é um conceito importante em orientação a objetos e, consequentemente na linguagem Java. Considerando esse conceito, assinale a única alternativa correta.

A

Em Java, chama-se hierarquia a relação entre dois objetos em que um controla o comportamento do outro.

B

Para a linguagem Java, uma classe que sobrescreva o método de sua superclasse promove a generalização dessa classe.

C

Ao se instanciar o objeto de uma classe derivada, pode-se dizer que ele pertence a um subtipo da superclasse Java desse objeto.

D

Em Java, quando um objeto possui duas superclasses, tem-se um caso de múltipla hierarquia.

E

Java não admite herança de classes em múltiplos níveis.

A alternativa C está correta.

A hierarquia de classes estabelece uma hierarquia de tipos, na qual as classes derivadas representam subtipos de suas superclasses. Logo, um objeto instanciado de uma classe derivada é, por extensão, pertencente a um subtipo da superclasse.

Explorando a hierarquia de herança

Entendido o que é herança e como Java a implementa, vamos explorar esse paradigma para compreender melhor a forma como a linguagem Java o aborda. Em outras palavras, queremos entender o que levou os criadores da linguagem a rejeitarem a ideia de herança múltipla, entender como ela se aplica às classes internas (*inner class*) de Java e conhecer o modificador de acesso default.

Explore neste vídeo o conceito de herança múltipla e as estratégias para contornar as limitações de sua aplicação em Java.

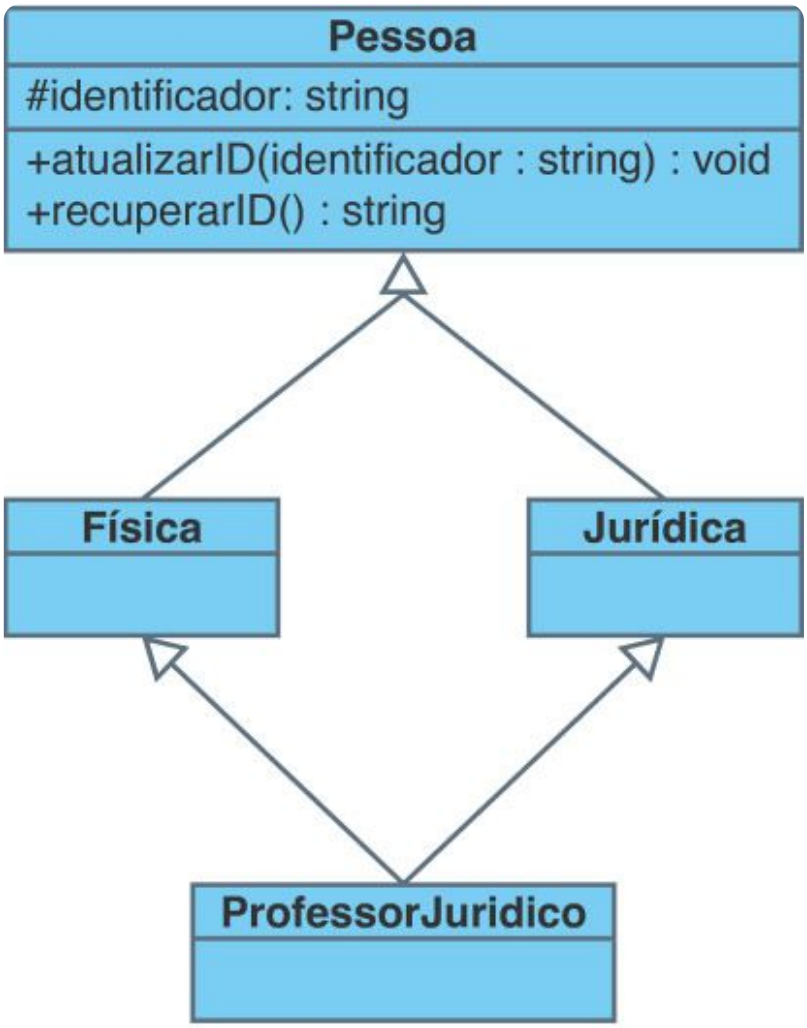
Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Problema da herança múltipla

O exemplo da imagem anterior pode ter lhe despertado alguns questionamentos. Nele, Aluno está especializando o método “atualizarID”, que é da superclasse Pessoa. Ao mesmo tempo, um objeto Aluno também é um tipo de Fisica, pois essa classe busca modelar o conceito de “pessoa física” (é razoável esperar que um aluno possua CPF além de seu código de matrícula). Será que essa situação não seria melhor modelada através de herança múltipla, com Aluno derivando diretamente de Pessoa e Física?

Já falamos que Java não permite herança múltipla e que é possível solucionar tais casos modificando a modelagem. Agora entenderemos o problema da herança múltipla que faz a Java evitá-la. Para isso, vamos modificar o modelo mostrado na imagem anterior, criando a classe ProfessorJuridico, que modela os professores que são contratados como pessoa jurídica. Essa situação é vista na próxima imagem. Nela, as classes Fisica e Juridica especializam o método “atualizarID” de Pessoa e legam sua versão especializada à classe ProfessorJuridico. Mas uma questão surge agora: quando “atualizarID” for invocado em um objeto do tipo ProfessorJuridico, qual das duas versões especializadas será executada? Essa é uma situação que não pode ser resolvida automaticamente pelo compilador. Por conta dessa ambiguidade, a imagem a seguir é também chamada de diamante da morte.



Diamante da morte

Linguagens que admitem herança múltipla deixam para o desenvolvedor a solução desse problema. Em C++, por exemplo, a desambiguação é feita fazendo-se um cast explícito para o tipo que se quer invocar. Entretanto, deixar a cargo do programador é potencializar o risco de erro, o que Java tem como meta evitar. Portanto, em Java a herança múltipla é proibida, obrigando a uma solução de modelagem distinta.

Modificador de acesso default

Se você revir os códigos que estudamos, perceberá que em todos os casos as classes foram declaradas públicas (todas são antecedidas pelo modificador public). Antes de continuarmos, mova a classe Pessoa para

pacote Gere (no mesmo projeto) e remova o modificador public na declaração da classe, de forma que as linhas 2 e 5 do Código 1 fiquem da seguinte forma:

Trecho do código 1 alterado:

```
java

...
package com.mycompany.Gere
...
...
class Pessoa
```

Agora tente executar o código abaixo.

Código 8: Testando o modificador default:

```
java

import com.mycompany.GereEscola.Pessoa;

class Derivada extends Pessoa {
    public Derivada ( String nome , String id ) {
        super ( nome , id );
    }
    public String getNome ( ) {
        return recuperarNome ( );
    }
}
public class Principal {
     public static void main ( String args [ ] ) {
        Derivada Der = new Derivada ( "Marcos" , "A1" );
        System.out.println ( Der.getNome() );
     }
}
```

Você notará que o código não executará. Retorne à declaração da classe Pessoa, reinsira o modificador public e tente executar o Código 8 novamente. O que ocorreu agora?

O Código 8 não pode ser executado quando a classe Pessoa não possuía o modificador public na declaração por causa do modificador default. Em Java, quando não se explicita a visibilidade da classe, ela é automaticamente definida como default. Esse modificador restringe a visibilidade da classe apenas o pacote ao qual a classe pertence.

No exemplo que vimos, as classes Pessoa e Derivada pertencem a pacotes diferentes — Gere e GereEscola, respectivamente. Logo, mesmo com a instrução de importação existente na linha 1 do Código 8, a classe Pessoa não é visível para a classe Derivada, a menos que possua o modificador public.

Veja a seguir um resumo da restrição imposta pelos modificadores de acesso de Java, que afetam também a herança. Note que os modificadores private e protected somente podem ser aplicados a classes aninhadas.

Modificador de acesso	Visibilidade			
	Na classe	No pacote	Fora do pacote, pela subclasse	Fora do pacote
private	sim	não	não	não
default	sim	sim	não	não
protected	sim	sim	sim	não

Modificador de acesso	Visibilidade			
	Na classe	No pacote	Fora do pacote, pela subclasse	Fora do pacote
public	sim	sim	sim	sim

Modificadores de acesso e visibilidade em Java
Marlos de Mendonça Corrêa

Herança e classes internas (inner class) de Java

Em Java, uma classe interna é uma classe declarada dentro (internamente a) de outra classe. É um conceito que permite aninhar classes. O Código 9 mostra um exemplo.

Código 9: Exemplo de classes internas em Java:

```
java

class Externa {
   void getPosClasse () {
       System.out.println( "Externa" );
   }
    class Intermediaria {
        void getPosClasse () {
            System.out.println( "Intermediaria" );
        }
        class Interna {
            void getPosClasse () {
                System.out.println( "Interna" );
            }
        }
    }
}
public class Principal {
    public static void main ( String args [ ] ) {
        Externa.Intermediaria.Interna in = new Externa().new Intermediaria().new
Interna();
        in.getPosClasse();
    }
}
```

O aninhamento de classes abre algumas possibilidades interessantes. A primeira, e talvez mais natural, é que as classes internas podem ser estendidas por outras classes internas. Essa herança se dá no escopo da classe, de maneira similar à que ocorre no escopo do pacote. Vejamos um exemplo!

Código 10: Extensão de uma classe interna por outra classe interna:

```
java

class Externa {
   void getPosClasse () {
       System.out.println( "Externa" );
   }
    class Intermediaria {
        void getPosClasse () {
            System.out.println( "Intermediaria" );
        }
        class Interna {
            void getPosClasse () {
                System.out.println( "Interna" );
            }
        }
    }
    class IntermediariaEstendida extends Intermediaria {
        void getPosClasse () {
            super.getPosClasse();
            System.out.println( "Intermediaria Estendida" );
        }
    }
}
public class Principal {
    public static void main ( String args [ ] ) {
        Externa.IntermediariaEstendida in = new Externa().new IntermediariaEstendida();
        in.getPosClasse();
    }
}
```

A execução desse código produz a saída a seguir.

```
java

--- exec:3.0.0:exec (default-cli) @ teste ---
Intermediaria
Intermediaria Estendida
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Como já dito, o modificador de acesso impacta a visibilidade da classe. Assim, se a linha 9 do Código 10 for alterada para “private class Interna {“, a classe mais interna (Interna) não será acessível pela classe IntermediariaEstendida. Assim, a tentativa de se fazer “Externa.IntermediariaEstendida.Interna in = new Externa().new IntermediariaEstendida().new Interna();” nesse caso irá gerar erro.

Outra possibilidade é estender a classe interna por uma classe fora do aninhamento, como mostrado no próximo exemplo.

Código 11: Extensão de uma classe interna por outra classe fora do aninhamento:

```
java
```

```
class Externa {
   void getPosClasse () {
       System.out.println( "Externa" );
   }
    class Intermediaria {
        void getPosClasse () {
            System.out.println( "Intermediaria" );
        }
        class Interna {
            void getPosClasse () {
                System.out.println( "Interna" );
            }
        }
    }
}
class NaoAninhada extends Externa.Intermediaria.Interna {
    public NaoAninhada ( ) {
        new Externa().new Intermediaria().super();
    }
    void getPosClasse () {
        super.getPosClasse();
        System.out.println( "Intermediaria Estendida" );
    }
}
public class Principal {
    public static void main ( String args [ ] ) {
        NaoAninhada na = new NaoAninhada ();
        na.getPosClasse();
    }
}
```

Eis a saída do Código 11.

```
java
```

```
--- exec:3.0.0:exec (default-cli) @ teste ---
Interna
Intermediaria Estendida
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Obviamente, da mesma forma que no caso anterior, o modificador de acesso afeta a acessibilidade da classe na herança. Contudo, se você avaliar, notará que os princípios de herança vistos anteriormente são os mesmos aplicados nos casos de aninhamento de classes.

Atividade 2

Linguagens OO permitem herança simples ou múltipla. Em relação a isso, marque a única alternativa correta.

A

A herança múltipla é um paradigma OO implementado em Java por meio da conversão explícita (cast).

B

Uma classe interna declarada como protegida pode ser acessada por uma classe em outro pacote que estenda a classe externa declarada pública.

C

Uma classe interna declarada como protegida não pode ser acessada por uma classe no mesmo pacote e fora do aninhamento que estenda a classe externa.

D

Todos os modificadores de acesso são aplicáveis à classe mais externa do aninhamento.

E

O aninhamento de classes é um tipo de herança específico de Java.

A alternativa B está correta.

Quando uma classe estende outra que é pública, seus membros são herdados com o modificador que os precede. Assim, se a classe base possui uma classe interna declarada como protegida (protected), essa classe será visível para a classe derivada, mesmo que esta esteja em pacote distinto.

Herança, subtipos e o princípio da substituição de Liskov

Nas seções anteriores, você pôde compreender melhor o mecanismo de herança e como ele afeta os objetos instanciados. Falamos sobre especialização e generalização, mostrando como tais conceitos funcionam dentro de uma hierarquia de classes. Você deve ter notado que o método “atualizarID” das classes Pessoa, Fisica e Juridica possuem a mesma assinatura. Isso está alinhado ao princípio de projeto por contrato (*design by contract*).

A partir de agora, você vai entender o que significa dizer que uma classe derivada é um subtipo da classe base e como isso impacta o desenvolvimento de software. Vamos apresentar brevemente o princípio de substituição de Liskov mostrando como ele se relaciona com o conceito de subtipo e herança: a chamada subtipagem comportamental forte — um caso particular de subtipagem comportamental.

Confira no vídeo a relação entre os conceitos de tipo e de classe, o conceito de subtipos e o princípio de substituição de Liskov.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Princípio da substituição de Liskov

No projeto por contrato, os métodos de uma classe definem o contrato que se estabelece com a classe ao consumir os serviços que esta disponibiliza. Assim, para o caso em questão, o contrato determina que para atualizar o campo “identificador” de um objeto, deve-se invocar o método “atualizarID”, passando-lhe um objeto do tipo “String” como parâmetro. E não se deve esperar qualquer retorno do método.

> Rigorosamente falando, subtipo e subclasse são conceitos distintos.

Sobre subtipo e subclasse podemos dizer que uma subclasse é estabelecida quando uma classe é derivada de outra, mas um subtipo tem uma restrição adicional. Para que uma subclasse seja um subtipo da superclasse, todas as propriedades da superclasse devem ser válidas na subclasse. Isso não ocorre para o caso que acabamos de descrever. Nas classes Pessoa, Fisica e Juridica, a propriedade de definição do identificador é a mesma: o campo “identificador” é definido a partir de um objeto “String” fornecido. Na classe Aluno o campo “identificador” é definido automaticamente.

A consequência imediata da mudança de propriedade feita pela classe Aluno é a modificação do contrato estabelecido pela superclasse. Com isso, um programador que siga o contrato da superclasse ao usar a classe Aluno terá problemas pela alteração no comportamento esperado. Essa situação nos remete ao princípio da substituição de Liskov (Liskov; Wing, 1994).



Comentário

O princípio da substituição foi primeiramente apresentado por Barbara Liskov em 1987 e faz parte dos chamados princípios SOLID (single responsibility, open-closed, Liskov substitution, interface segregation and dependency inversion). Ele pode ser encontrado com diversos enunciados, sempre afirmando a substitutibilidade de um objeto da classe base pela classe derivada, sem prejuízo para o funcionamento do software. Podemos enunciá-lo da seguinte forma: seja um programa de computador P e os tipos B e D, tal que D deriva de B. Se D for subtipo de B, então qualquer que seja o objeto b do tipo B ele pode ser substituído por um objeto d do tipo D sem prejuízo para P.

Voltando ao exemplo em análise, as classes Fisica e Juridica estão em conformidade com o contrato da classe Pessoa, pois não o modificam, embora possam adicionar outros métodos. Assim, em todo ponto do programa no qual um objeto de Pessoa for usado, objetos de Fisica e Juridica podem ser aplicados sem quebrar o programa, mas não acontece a mesma situação com objetos do tipo Aluno.

Atividade 3

Considere duas classes chamadas Base e Derivada, tais que a primeira é superclasse da última. A classe Derivada atende ao princípio da substituição de Liskov. Com isso em mente, analise as afirmativas a seguir.

I. Derivada não pode sobrescrever os métodos protegidos de Base.

II. Todos os métodos públicos e protegidos de Derivada devem ter a mesma assinatura que os de Base.

III. Todos os métodos públicos e protegidos de Base que forem redefinidos em Derivada devem ter a mesma assinatura.

Está correto o que se afirma em:

A

I.

B

II.

C

III.

D

I e III.

E

I e II.

A alternativa C está correta.

O princípio da substituição de Liskov afirma que a classe Base deve poder ser substituída por Derivada sem que o programa deixe de funcionar. Assim, é necessário que os métodos de Base redefinidos por Derivada tenham a mesma assinatura.

Entendendo a hierarquia de herança

É hora de aplicarmos os conhecimentos obtidos! Faremos isso de forma prática de modo que você entenda o uso dos conceitos vistos. Para isso, trabalharemos com classes implementadas em dois pacotes distintos. Por questões didáticas e para facilitar a compreensão, chamaremos ao primeiro pacote de PctAlfa e ao segundo de PctBravo.

O pacote PctAlfa conterá as classes Principal e Execucao. A classe Principal modela o comportamento do programa e é a classe que é chamada na sua execução. Já a classe Execucao será usada para estender as demais classes, permitindo-nos explorar o conceito de herança.

O pacote PctBravo conterá duas classes públicas, Base e Estendida, e as classes aninhadas Externa e Interna. Essas classes possuirão comportamentos simples, mas representativos, possibilitando explorarmos a herança.

Descubra no vídeo como usar o conceito de herança para modificar o comportamento de métodos usados ao longo de um código.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Como é mandatório em Java, todas as classes públicas devem estar definidas em um arquivo de mesmo nome. No nosso exemplo, são públicas as classes Principal, Base e Estendida. Além disso, para o caso em questão, teremos dois pacotes: PctAlfa e PctBravo. Assim, você deve inicialmente criar um projeto em sua IDE e, posteriormente, criar ambos os pacotes. Lembre-se de importar em PctAlfa o pacote PctBravo. Depois crie as classes que deverão ter, inicialmente, os códigos mostrados a seguir.

Código 12: Classe pública Base e classes aninhadas Externa e Interna:

```
java

package com.mycompany.PctBravo;
class Externa {
    private String id;
    public Externa ( ) {
        id = "PctBravo.Externa";
        System.out.println( "PctBravo.Externa instanciada." );
    }
    public String getId () {
       return id;
    }
    class Interna {
        public Interna ( ) {
        id = "PctBravo.Externa.Interna";
        System.out.println( "PctBravo.Externa.Interna instanciada." );
        }
        public String getId () {
            return id;
        }
    }
}
public class Base extends Externa.Interna {
    private String id;
    public Base ( ) {
        new Externa().super();
        id="PctBravo.Base";
        System.out.println( "PctBravo.Base instanciada." );
    }
}
```

Declaração da Classe Estendida:

Código 13: Classe Estendida:

```
java

package com.mycompany.PctBravo;
public class Estendida extends Externa.Interna {
    private String id;
    public Estendida ( ) {
        new Externa().super();
        id="PctBravo.Estendida";
        System.out.println( "PctBravo.Estendida instanciada." );
    }
}
```

Código 14: Classe pública Principal e classe Execucao:

```
java

package com.mycompany.PctAlfa;
import com.mycompany.PctBravo.*;
class Execucao {
    private String id;
    public Execucao ( ) {
        id="PctAlfa.Execucao";
        System.out.println( "PctAlfa.Execucao instanciada." );
    }
}
public class Principal {
    static Execucao ex;
    static Base bs;
    public static void main(String[] args) {
        ex = new Execucao ();
        bs = new Base ();
        System.out.println ( bs.getId() );
    }
}
```

Agora execute o programa. Confira o que você deve obter como saída!

```
java

--- exec:3.1.0:exec (default-cli) @ Tema2_Modulo_1A ---
PctAlfa.Execucao instanciada.
PctBravo.Externa instanciada.
PctBravo.Externa.Interna instanciada.
PctBravo.Base instanciada.
PctBravo.Externa.Interna
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Vamos entender o que ocorreu. Na linha 14 do Código 14, ocorre a instanciação da classe Execucao, levando à impressão da linha 2 da saída. Já na linha 15 do Código 14, ocorre a instanciação da classe Base, que está em outro pacote. Como a classe é pública e foi importada através do comando import (linha 2 do Código 14), ela está disponível e pode ser instanciada. Isso desencadeia a execução do construtor de Base.

A classe Base estende a classe Interna, que está aninhada na classe Externa (linha 11 do Código 12). Embora Interna seja uma classe membro de Externa, Base está estendendo apenas Interna. Entretanto, como Interna é uma *inner class*, sua declaração fica sombreada pela classe que a engloba (Externa). Assim, ela só é acessível através de referência completamente qualificada (Externa.Interna). Dessa forma, a instrução “public class Base extends Externa.Interna” (linha 21 do Código 12) instrui o compilador como localizar a classe estendida.

Voltando à execução, observamos que no construtor de Base em que ocorre a instanciação Interna (linha 24 do Código 12) há um encadeamento de chamadas. Veja que o método “super”, usado para se referir ao construtor da classe pai, aparece associado ao construtor da classe Externa. Vamos entender o que ocorre.

A classe Interna é membro de Externa. Isso significa que ela existe no escopo de Externa. Ou seja, para que Interna seja instanciada, é necessária uma instância de Externa. Assim, apesar de Base estender apenas Interna, é preciso, antes de instanciá-la, instanciar a classe que a engloba, o que é feito com o operador new e o construtor de Externa. Ou seja, “new Externa ()” é uma instância de externa. Por isso vemos a linha 3 da saída, gerada pelo construtor de Externa.

Como “new Externa ()” é um objeto, podemos agora invocar o construtor de sua classe aninhada Interna, o que fazemos através do operador ponto (.), como em qualquer método. Isso leva à instanciação de Interna,

como se vê na linha 4 da saída. Em seguida, completa-se a instanciação da classe Base, provocando a saída vista na linha 5.

Seguindo a execução, o compilador executa a linha 16 do Código 14. Observe que aqui é invocado o método “getId”. A classe Base não possui esse método, mas ele foi herdado de Interna e assim está disponível. A sua execução imprime a linha 6 da saída que, como vemos, comprova que o método executado foi o “getId” de Interna.

Por fim, podemos ver que embora a classe Estendida não tenha sido instanciada, o seu código foi compilado sem problemas. Agora faremos uma pequena modificação, alternado a linha 11 do Código 12, inserindo no início o modificador de acesso protected. Também vamos modificar a classe Execucao para que ela estenda Externa. Então passamos a ter uma hierarquia como a mostrada na próxima imagem. Tente compilar o código modificado.

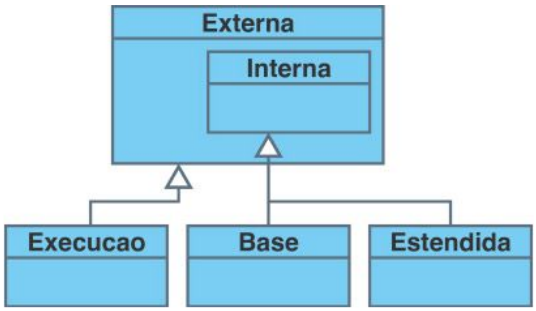


Diagrama de classes

O que aconteceu? Você sabe explicar o porquê? Se não, revise as lições e busque entender a razão para isso.

Vamos fazer mais uma alteração. Você deve tornar a classe Externa pública. Isso significa movê-la para um arquivo de mesmo nome, dentro do pacote PctBravo. Observe a seguir como ficará o código da classe Externa.

Código 15: Classe Externa modificada:

```
java

package com.mycompany.PctBravo;
public class Externa {
    private String id;
    public Externa ( ) {
        id = "PctBravo.Externa";
        System.out.println( "PctBravo.Externa instanciada." );
    }
    public String getId () {
       return id;
    }
    protected class Interna {
        public Interna ( ) {
            id = "PctBravo.Externa.Interna";
            System.out.println( "PctBravo.Externa.Interna instanciada." );
        }
        public String getId () {
            return id;
        }
    }
}
```

Altera o código de Execução e Principal para refletir o mostrado no Código 16.

Código 16: Classes Execucao e Principal modificadas:

```
java

package com.mycompany.PctAlfa;
import com.mycompany.PctBravo.*;

class Execucao extends Externa {
    private String id;
    Externa.Interna exint;
    public Execucao ( ) {
        exint = new Externa().new Interna();
        id="PctAlfa.Execucao";
        System.out.println( "PctAlfa.Execucao instanciada." );
    }
}
public class Principal {
    static Execucao ex;
    public static void main(String[] args) {
        ex = new Execucao ();
        System.out.println ( ex.getId() );
    }
}
```

Ao executar o programa você deve obter a saída a seguir.

```
java

--- exec:3.1.0:exec (default-cli) @ Tema2_Modulo_1A ---
PctBravo.Externa instanciada.
PctBravo.Externa instanciada.
PctBravo.Externa.Interna instanciada.
PctAlfa.Execucao instanciada.
PctBravo.Externa
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Veja a linha 4 do Código 16. Nessa linha, estamos fazendo com que a classe Execucao estenda a classe Externa que engloba Interna. Como Interna é protegida, ela não pode ser estendida fora do pacote onde foi definida, mas pode ser acessada. Para isso, na linha 8 nós instanciamos a classe Interna, o que imprime a saída da linha 4.



Comentário

Você pode e deve fazer mudanças no código e testar outras possibilidades. Isso ajuda a sedimentar os conceitos. Lembre-se, contudo, de que os modificadores de acesso têm papel fundamental na hierarquia de herança. São eles que regulam a visibilidade dos membros de uma classe.

Faça você mesmo!

Ao implementar um programa, o desenvolvedor precisa estender uma classe que se situa em outro pacote. Confira os trechos de código pertinentes.

```
1. package com.mycompany.PctAlfa;
2. import com.mycompany.PctBravo.*;
3.
4. class Derivada extends Externa {
5. ...// código omitido
6. }
```

```
1. package com.mycompany.PctBravo;
2. class Externa {
3.     private String id;
4.     public Externa ( ) {
5.         id = "PctBravo.Externa";
6.         System.out.println( "Funciona." );
7.     }
```

No entanto, ao compilar o programa foi gerado erro, indicando que a classe Externa não era accessível. Sabendo que o erro está em uma das linhas de código mostradas, assinale a única opção que fará o programa funcionar corretamente.

A

Aplicar o modificador default à classe Externa.

B

Declarar o construtor da classe Externa como protected.

C

Alterar a declaração da classe Derivada incluindo a referência completa de Externa (class Derivada Extends com.mycompany.PctBravo.Externa).

D

Aplicar o modificador public à classe Externa.

E

Aplicar o modificador protected à classe Externa.

A alternativa D está correta.

Para que uma classe seja diretamente extensível por outra pertencente a outro pacote, a classe base deve ser pública.

Hierarquia de coleção

Frameworks são ferramentas muito úteis e que agilizam bastante o processo de desenvolvimento. Por serem estruturas sobre as quais o software pode ser implementado, o software pode se valer de códigos testados e eficientes. A Coleção de Java é um ótimo exemplo. Esse framework oferece uma interface padrão que facilita o reuso de código e oferece estruturas de dados e algoritmos, o que auxilia o trabalho de desenvolvimento.

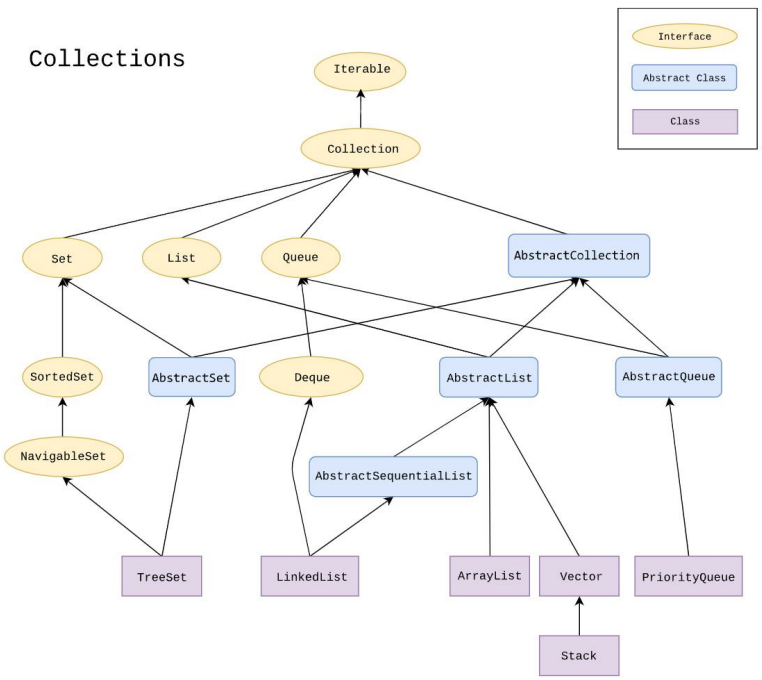
A partir de agora veremos essa importante ferramenta e como ela pode ser empregada no desenvolvimento. A hierarquia de coleção é ampla e certamente esse assunto não pode ser esgotado aqui. Nosso objetivo é mostrar a sua potencialidade e utilidade. Esse conhecimento contribuirá para torná-lo um profissional com recursos. Após esse estudo, você terá mais uma ferramenta à disposição para utilizar, quando necessário.

Conheça neste vídeo um importante framework disponível na linguagem Java: o framework collection e suas especializações.

> **Conteúdo interativo**
>
> Acesse a versão digital para assistir ao vídeo.

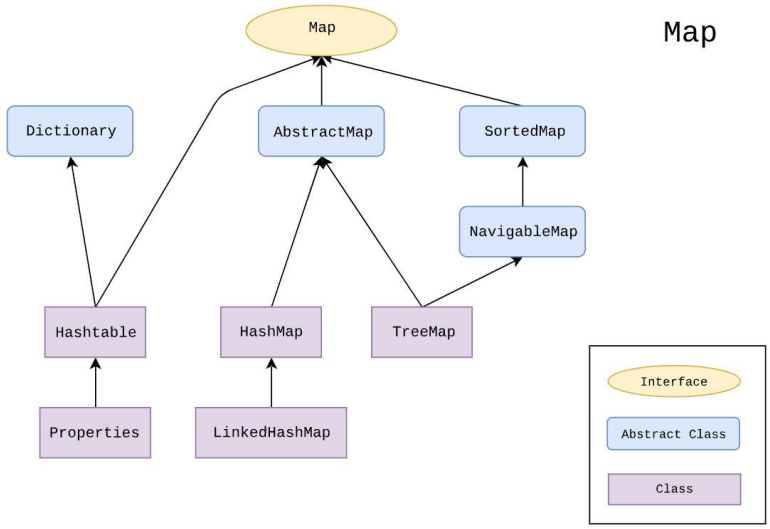
Coleções ou containers são um conjunto de classes e interfaces Java chamados de Java Collections Framework. Essas classes e interfaces implementam estruturas de dados comumente utilizadas para agrupar múltiplos elementos em uma única unidade. Sua finalidade é armazenar, manipular e comunicar dados agregados (ORACLE AMERICA INC., s.d.).

Entre as estruturas de dados implementadas, temos Set, List, Queue e Deque. Embora a estrutura Map não seja verdadeiramente uma coleção, ela também é provida pela Java Collections Framework. Isso significa que o framework é formado por duas árvores de hierarquia, uma correspondente às entidades derivadas de Collection e outra formada pela Map, como vemos nas imagens a seguir!



Hierarquia de herança – Collections

Agora que você já viu a hierarquia de herança – Collections, observe a hierarquia de herança – Map.



Hierarquia de herança – Map

Observe nas imagens que essas estruturas são implementadas aplicando os conceitos de herança vistos anteriormente. Por exemplo, SortedSet e SortedMap são especializações de Set e Map, respectivamente, as quais incorporam a ordenação de seus elementos.

A interface Collection possui os comportamentos mais genéricos (assim como Map na sua hierarquia). Por exemplo, o mecanismo para iterar pela coleção pode ser encontrado nela. Ela possui, também, métodos que fornecem comportamentos genéricos. É um total de 15 métodos, mostrados a seguir.

Método	Comportamento
add(Object e)	Insere um elemento na coleção.
addAll(Collection c)	Adiciona os elementos de uma coleção em outra.
clear()	Limpa ou remove os elementos de uma coleção.
contains(Object c)	Verifica se um dado elemento está presente na coleção.
containsAll(Collection c)	Verifica se todos os elementos de uma coleção estão presentes em outra.
equals(Object e)	Verifica se dois objetos são iguais.
hashCode()	Retorna o hash de uma coleção.
isEmpty()	Retorna verdadeiro se a coleção estiver vazia.
Iterator()	Retorna um iterador.
remove(Object e)	Remove um determinado elemento da coleção.
removeAll(Collection c)	Remove todos os elementos da coleção.
retainAll(Collection c)	Remove todos os elementos de uma coleção exceto os que correspondem a “c”.
size()	Retorna o número de elementos da coleção.
toArray()	Retorna os elementos de uma coleção em um vetor.

Método	Comportamento
Objetct [ ] toArray (Object e)	Retorna um vetor que contém todos os elementos da coleção que o invoca.

Métodos da interface Collection
Marlos de Mendonça Corrêa

As interfaces Set, List, Queue e Deque possuem comportamentos especializados conforme as seguintes particularidades de cada tipo:

Set

Representa conjuntos e não admite elementos duplicados.

List

Implementa o conceito de listas e admite duplicidade de elementos. É uma coleção ordenada que permite o acesso direto ao elemento armazenado, assim como a definição da posição em que está armazenado.

Queue

Implementa algo mais genérico do que uma fila. Pode ser usada para criar uma fila (FIFO), mas também pode implementar uma lista de prioridades, na qual os elementos são ordenados e consumidos segundo a prioridade, e não na ordem de chegada. Essa coleção admite a criação de outros tipos de filas com outras regras de ordenação.

Deque

Implementa a estrutura de dados conhecida como Deque *(double ended queue).* Pode ser usada como uma fila (FIFO) ou uma pilha (LIFO). Admite a inserção e retirada em ambas as extremidades.

Nas hierarquias de coleções mostradas, as interfaces e classes abstratas definem o contrato que deve ser seguido pelas classes que lhes são derivadas. Essa abordagem atende ao princípio de substituição de Liskov, garantindo que as derivações constituam subtipos. As classes que implementam os comportamentos desejados são vistas em roxo nas figuras.

Saiba mais

O Java Collections Framework é uma ferramenta útil e que não poderia ser esgotada neste espaço. No entanto, como programador, você deve saber todas as possibilidades que ele oferece. Você encontrará informações abundantes na internet, mas o ponto de partida ideal é a documentação oficial da linguagem, disponível em docs.oracle.com/javase/8/docs/technotes/guides/collections/overview.html.

Atividade 1

Estruturas de dados são mecanismos fundamentais para a manipulação de dados e possuem impacto na performance de um software. Java fornece implementações de algumas estruturas através do Java Collections Framework. Sobre o assunto, marque a única opção correta.

A

A herança múltipla é um paradigma OO implementado em Java por meio da conversão explícita (cast).

B

Uma classe interna declarada como protegida pode ser acessada por uma classe em outro pacote que estenda a classe externa declarada pública.

C

Uma classe interna declarada como protegida não pode ser acessada por uma classe no mesmo pacote e fora do aninhamento que estenda a classe externa.

D

Todos os modificadores de acesso são aplicáveis à classe mais externa do aninhamento.

E

O aninhamento de classes é um tipo de herança específico de Java.

A alternativa D está correta.

Filas podem ser implementadas por Queue e Deque.

Tipos estáticos e dinâmicos

Em uma rápida busca pela internet, você encontra o emprego equivocado dos conceitos de tipos estáticos e dinâmicos. Muitos associam a vinculação dinâmica na invocação de métodos dentro de hierarquias de classe à tipagem dinâmica, mas isso não está correto. Trata-se, nesse caso, de vinculação dinâmica (*dynamic binding*). A partir de agora, você vai entender melhor!

Assista ao vídeo e compreenda os conceitos de tipagem estática e dinâmica, e como eles são aplicados na linguagem Java.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Para iniciarmos nesse assunto, você precisa entender o que é tipagem dinâmica e estática. Tipagem é atribuir um tipo a uma variável. Um tipo de dado é um conjunto fechado de elementos e propriedades que são afetas aos elementos. Por exemplo, quando definimos uma variável como sendo do tipo inteiro, o compilador gera comandos para alocação de memória suficiente para guardar o maior valor possível para esse tipo.

Lembremos que o tipo inteiro em Java — e em qualquer linguagem — é uma abstração limitada do conjunto dos números inteiros, que é infinito. Além disso, o compilador é capaz de operar com os elementos do tipo “int”. Ou seja, adições, subtrações e outras operações são realizadas sem necessidade de implementar o comportamento. Situação semelhante vale para os demais tipos primitivos, como “String”, “float” e outros.

Há duas formas de tipar uma variável, veja!

Tipagem estática		Tipagem dinâmica
Quando o tipo é determinado em tempo de compilação.		Quando o tipo é determinado em tempo de execução.

Observe que não importa se o programador determinou o tipo. Se o compilador for capaz de inferir esse tipo durante a compilação, então trata-se de tipagem estática. A tipagem dinâmica ocorre quando o tipo só pode ser identificado durante a execução do programa.

Um exemplo de linguagem que emprega tipagem dinâmica é a JavaScript. O Código 17 mostra um trecho de programa em JavaScript que exemplifica a tipagem dinâmica.

Código 17: Tipagem dinâmica em JavaScript:

```
javascript

var a;
console.log('O tipo de a eh: '+ typeof(a));
a = 1;
console.log('O tipo de a eh: '+ typeof(a));
a = "Palavra";
console.log('O tipo de a eh: '+ typeof(a));
a = false;
console.log('O tipo de a eh: '+ typeof(a));
a = [];
console.log('O tipo de a eh: '+ typeof(a));
```

Confira a saída da execução desse código.

```
javascript

O tipo de a eh: undefined
O tipo de a eh: number
O tipo de a eh: string
O tipo de a eh: boolean
O tipo de a eh: object
```

A mesma variável (“a”) assumiu ao longo da execução diferentes tipos. Observando a linha 1 do Código 17, vemos que na declaração o compilador não tinha qualquer informação que lhe permitisse inferir o tipo de dado de “a”. Esse tipo só pode ser determinado na execução, nas linhas 3, 5, 7 e 9. Todas essas linhas alteram sucessivamente o tipo atribuído a “a”.

Curiosidade

Você deve ter em sua mente que a linguagem de programação Java é estaticamente tipada, o que significa que todas as variáveis devem ser primeiro declaradas (tipo e nome) e depois utilizadas (ORACLE AMERICA INC., 2021b)

Uma possível fonte de confusão é a introdução da instrução “var” a partir da versão 10 da Java. Esse comando, aplicável apenas às variáveis locais, instrui o compilador a inferir o tipo de variável. Entretanto, o tipo é determinado ainda em tempo de compilação e, uma vez determinado, ele não pode ser modificado. A Java exige que, no uso de “var”, a variável seja inicializada, o que permite ao compilador determinar seu tipo. O Código 18 mostra o emprego correto de “var”, enquanto o Código 19 gera erro de compilação. Ou seja, “var” em Java não possui a mesma semântica que “var” em JavaScript. São situações distintas e, em Java, trata-se de tipagem estática.

Código 18: Uso correto de "var" em Java — tipagem estática:

```
java

...
var a = 0;
a = a + 1;
...
```

Código 19: Uso errado de "var" em Java:

```
java

...
var a = 0;
a = “Palavra”
...
```

Uma outra possível fonte de confusão diz respeito à vinculação dinâmica (*dynamic binding*) de método. Para ilustrar essa situação vamos adicionar o método “retornaTipo” às classes Pessoa, Fisica e Juridica, cujas respectivas implementações são vistas nos códigos a seguir. Vamos começar pelo Código 20 com a implementação do método à classe Pessoa.

Código 20: "retornaTipo" de Pessoa:

```
java

public String retornaTipo ( ) {
    return null;
}
```

Agora a implementação do método às classes Fisica e Juridica.

Código 21: "retornaTipo" de Juridica:

```
java

public String retornaTipo ( ) {
    return “Fisica”;
}
```

Código 22: Uso errado de "var" em Java:

```
java

public String retornaTipo ( ) {
    return “Juridica”;
}
```

Se lembrarmos que as classes Fisica e Juridica especializam Pessoa, ficará óbvio que ambas substituem o método “retornaTipo” de Pessoa por versões especializadas. O Código 23 mostra um trecho de código da classe principal do programa.

Código 23: "retornaTipo" de Pessoa:

```
java

public class Principal {
    private static Pessoa grupo [];
    ...
    grupo = new Pessoa [3];
    grupo [0] = new Fisica ( "Marco Antônio" , data_nasc , null , null , "Brasil" , "Rio
de Janeiro" );
    grupo [1] = new Juridica ("Escola Novo Mundo Ltda" , data_nasc , null , null ,
"Brasil" , "Rio de Janeiro");
    grupo [2] = new Pessoa ( null , null , null , null , "Brasil" , "Rio de Janeiro");
    for ( int i = 0 ; i <= 2 ; i++ )
    System.out.println( grupo [i].retornaTipo ( ) );
    ...
}
```

A execução desse código produz como saída:

```
java

Fisica
Juridica
null
```

O motivo disso é a vinculação dinâmica de método. O vetor “grupo []” é um arranjo de referências para objetos do tipo Pessoa. Como Fisica e Juridica são subclasses de Pessoa, os objetos dessas classes podem ser referenciados por uma variável que guarda referência para a superclasse. Isso é o que ocorre nas linhas 5 e 6 do Código 23.

A vinculação dinâmica ocorre na execução da linha 9. O compilador não sabe *a priori* qual objeto será referenciado nas posições do vetor, mas, durante a execução, ele identifica o tipo do objeto e vincula o método apropriado. Essa situação, entretanto, não se configura como tipagem dinâmica, pois o tipo do objeto é usado para se determinar o método a ser invocado. Nesse caso, devemos lembrar também que Java oculta o mecanismo de ponteiros, e uma classe definida pelo programador não é um dos tipos primitivos. Então “grupo []” é, como dito, um vetor de referências para objetos do tipo Pessoa e isso se mantém mesmo nas

linhas 5 e 6 (as subclasses são um tipo da superclasse). Essa é uma diferença sutil, mas significativa, com relação ao mostrado no Código 17.

> Em resumo, essa situação não é tipagem dinâmica, pois o vetor “grupo []” não alterou seu tipo, qual seja, referência para objetos do tipo Pessoa.

Atividade 2

Considere o trecho de código a seguir no qual Base é superclasse de Derivada1 e Derivada1 é superclasse de Derivada2.

```
1. Base obj1 = new Base ();
2. Base obj2 = new Derivada1 ();
3. Base obj3 = new Derivada2 ();
4. Derivada1 obj4 = new Derivada1();
```

Julgue as afirmações.

I. obj1 e obj2 são referências para objetos do tipo Base.

II. obj2 e obj3 são referências para objetos do mesmo tipo.

III. obj2 e obj4 são referências para objetos do tipo Derivada1.

Está correto o que se afirma em:

A

I.

B

II.

C

III.

D

I e II.

E

II e III.

A alternativa C está correta.

As variáveis obj1, obj2 e obj3 são todas do tipo Base e obj4 é do tipo Derivada1. Porém os objetos instanciados nas linhas 1, 2 e 3 são respectivamente do tipo Base, Derivada1 e Derivada2. Na linha 4, o objeto é do tipo Derivada1. Como Derivada1 e Derivada2 são subtipos de Base, o compilador identifica que a associação não contraria o tipo definido para a variável e permite que elas referenciem os objetos criados. Assim, obj2 é do tipo Base, mas é referência para um objeto do subtipo Derivada1.

Hierarquia de herança na prática

É chegada a hora de consolidar o aprendizado por meio de uma situação prática. Para isso, vamos utilizar um exemplo simples, porém com a aplicação de diversos conceitos estudados. Se você tiver alguma dificuldade de entendimento, retorne às sessões anteriores e faça isso tantas vezes quantas forem necessárias, até que consiga compreender claramente como o código funciona.

Nessa prática, vamos utilizar essencialmente três classes – Principal, PessoaFisica e Aluno. Usaremos também uma classe ComparaPessoa, que detalharemos posteriormente. Trabalharemos com o mínimo de funcionalidades, suficientes apenas para ilustrar a aplicação de conceitos.

A classe Principal implementa a interatividade do programa. A classe Aluno é uma subclasse de PessoaFisica que contém o atributo “matricula”. Nosso programa vai criar e manter um registro de alunos. Para isso, vamos nos valer do framework de coleção de Java (Collection) e escolheremos uma estrutura de dados que nos ajude a manter um grande volume de registros, pensando principalmente na busca.

Alterar o código, explorando como as modificações se comportam, é uma excelente ferramenta de aprendizagem; por isso, fique à vontade para brincar! O código que mostramos possui várias limitações. Uma sugestão é transformá-lo num gerenciador de registros real, expandindo-o para também lidar com pessoas jurídicas. Se você decidir fazer isso, aplicará inclusive conceitos de OO.

Descubra neste vídeo como utilizar a herança para implementar códigos que especializam a classe Pessoa.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Vejamos os códigos das classes. Primeiro vamos observar as classes PessoaFisica (Código 24) e ComparaPessoa (Código 25).

Código 24: Classe PessoaFisica:

```
java

public class PessoaFisica {
    protected String CPF;
    protected String nome;

    public PessoaFisica ( String nome , String CPF ) {
        this.nome = nome;
        setCPF ( CPF );
    }
    protected void setNome ( String nome ) {
        this.nome = nome;
    }
    protected String getNome ( ) {
        return nome;
    }
    protected void setCPF ( String CPF ) {
        if ( validarCPF ( CPF ) )
            this.CPF = CPF;
        else
            System.out.println ( "CPF invalido!" );
    }
    protected String getCPF ( ) {
        return CPF;
    }
    private boolean validarCPF ( String CPF ) {


        //verifica sequência de dígitos iguais e tamanho (11 dígitos)
        if ( ! ( CPF.chars ( ).allMatch ( Character::isDigit ) ) ) {
            System.out.println ( "CPF possui caracteres não numéricos!" );
            return false;
        }
        else if ( ( CPF.length() != 11 ) || CPF.matches( "[0]{11}|[1]{11}|[2]{11}|[3]{11}|
[4]{11}|[5]{11}|[6]{11}|[7]{11}|[8]{11}|[9]{11}" ) )
            return false;
        return true;
    }
}
```

Vejamos a implementação da Classe ComparaPessoa.

Código 25: Classe ComparaPessoa:

```
java

//imports
import java.util.Comparator;

    public class ComparaPessoa implements Comparator {
    @Override
    public int compare ( PessoaFisica obj1 , PessoaFisica obj2 )
    {
        return obj1.getNome ().compareToIgnoreCase ( obj2.getNome ( ) ); //retorna zero
se os nomes são iguais. Desconsidera a caixa.
    }
}
```

A classe PessoaFisica modela comportamentos básicos, como os métodos para definir e recuperar atributos (nome e CPF). Repare que ao definir o CPF, ele é submetido a uma validação, mas apenas quanto ao tamanho e repetição de números. Isso foi feito por razões didáticas, assim você pode testar o código sem se preocupar em usar um CPF válido, que não é o nosso objetivo aqui.



Dica

Para validar o CPF, faça uma busca na internet, você encontrará diversas implementações de checagem de CPF!

A classe ComparaPessoa é mais interessante, pois ela concretiza a classe abstrata Comparator, sobrescrevendo o método “compare”. O termo PessoaFisica indica que estamos trabalhando com uma instância de concretização para objetos desse tipo. Assim, acabamos de prover um método de comparação para objetos do tipo PessoaFisica. Isso é necessário, pois a estrutura de dados que vamos utilizar demanda uma maneira para comparar objetos que não são do tipo primitivo. Por exemplo, se a estrutura fosse armazenar String, int ou double, isso não seria preciso, mas é obrigatório para tipos definidos pelo usuário.

A classe Aluno (Código 26) estende a classe PessoaFisica incluindo o atributo “matricula” e seus respectivos métodos de definição e recuperação. Adicionalmente, ela provê uma versão especializada de “toString”.

Código 26: Classe ComparaPessoa:

```
java

public class Aluno extends PessoaFisica {
    String matricula;

    public Aluno ( String nome , String CPF , String matricula ) {
        super ( nome , CPF );
        this.matricula = matricula;
    }
    public void setMatricula ( String matricula ) {
        this.matricula = matricula;
    }
    public String getMatricula ( ) {
        return matricula;
    }
    @Override
    public String toString () {
        return String.format("ObjID: %s => Nome:%s | CPF:%s | Matricula: %S\n" ,
Integer.toHexString ( this.hashCode() ) , nome , CPF , matricula );
    }
}
```

Vejamos agora a classe Principal (Código 27) e o funcionamento do programa.

Código 27: Classe ComparaPessoa:

```
java

//imports
import java.util.Scanner;
import java.util.SortedSet;
import java.util.TreeSet;

public class Principal {
    //Atributos
    private static SortedSet cj_ordenado;
    public static void main(String[] args) {
        cj_ordenado = new TreeSet ( new ComparaPessoa () );
        cj_ordenado.add ( new Aluno ( "Martins" , "12345678912" , "M1" ) );
        cj_ordenado.add ( new Aluno ( "Paula" , "12345678912" , "M2" ) );
        cj_ordenado.add ( new Aluno ( "Carla" , "36925814774" , "M3" ) );
        cj_ordenado.add ( new Aluno ( "Martins" , "45678912336" , "M4" ) );
        System.out.println( cj_ordenado );
    }
}
```

Se observarmos a linha 8, veremos que estamos usando uma estrutura SortedSet do framework Collection de Java. Da mesma forma que na classe ComparaPessoa, a expressão indica que estamos trabalhando com uma instância de SortedSet especializada para objetos do tipo Aluno. A linha 8 apenas declara a estrutura que é, de fato, inicializada na linha 10, mas na 8 estamos chamando TreeSet. Se você retornar à Imagem 4, verá que SortedSet é uma interface e TreeSet é uma das classes que a concretiza. É ela que implementa o comportamento. Essa classe implementa uma estrutura de dados do tipo árvore que garante tempo log (n) para as operações de inserção e remoção (ORACLE INC., S.d.).

Outra coisa que você deve reparar é que estamos passando no construtor de TreeSet (linha 10) um objeto do tipo ComparaPessoa. É através dessa referência que a estrutura TreeSet saberá como comparar os objetos. A comparação (linha 8 do Código 16) verifica as Strings que formam os nomes e retorna ou valor negativo (obj1 < obj2), ou zero (obj1 = obj2), ou valor positivo (obj1 > obj2). Veja que ComparaPessoa define o comportamento para objetos do tipo PessoaFisica. Mas como Aluno é um subtipo de PessoaFisica, esse método pode ser aplicado a objetos do tipo Aluno.

A saída da execução pode ser vista a seguir.

```
java

--- exec-maven-plugin:3.1.0:exec (default-cli) @ Tema2_Modulo_1 ---
[ ObjID: 4e50df2e => Nome: Carla | CPF: 36925814774 | Matricula: M3
, ObjID: 6e8cf4c6 => Nome: Martins | CPF: 12345678912 | Matricula: M1
, ObjID: 12edcd21 => Nome: Paula | CPF: 12345678912 | Matricula: M2 ]
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Observe agora dois pontos interessantes. Em primeiro lugar, vemos que os objetos foram impressos em ordem alfabética crescente. Isso ocorreu justamente pela forma como implementamos a comparação no Código 25 (classe ComparaPessoa). Nossa comparação baseou-se no atributo “nome”, o que o transformou no critério de ordenação.

Outro ponto é que fizemos quatro inserções, entretanto, ao imprimir os objetos da coleção, tivemos apenas três resultados. Se prestarmos atenção, veremos que o objeto Aluno ("Martins" , "45678912336" , "M4") não aparece. Ele foi nossa última inserção (linha 14 do Código 27). Na verdade, já existia na coleção um objeto com nome “Martins”, foi nossa primeira inserção. TreeSet não permite objetos com valor-chave repetidos, mas não gerou nenhuma mensagem de erro. Esse tratamento deve ser feito pelo programador.

Você deve ter notado que vários conceitos estão presentes nesse exemplo. Sugerimos que você modifique o código, para que ele faça a gestão de registros. Avalie usar outras estruturas do framework Collection e não se furte a experimentar diferentes abordagens. Você certamente aprenderá bastante.

Saiba mais

Conhecer as diferentes estruturas de disponíveis no framework Java Collection e saber como usá-las certamente lhe será muito útil. A internet está repleta de informações a esse respeito, complementadas com exemplos. Mas um excelente ponto de partida é a própria documentação da linguagem, disponível livremente no site docs.oracle.com/en/java/index.html.

Faça você mesmo!

Um programador deseja entender o funcionamento da estrutura ArrayList provida pelo framework Collection de Java. Para isso, ele executou o pequeno trecho de código a seguir.

```java
1. //imports
2. import java.util.ArrayList;
3. import java.util.Comparator;
4. import java.util.List;
5.
6. public class Principal implements Comparator {
7.     public static void main ( String args [ ] ) {
8.     List lista_cores = new ArrayList<>();
9.         lista_cores.add ( "Branco" );
10.         lista_cores.add ( "azul" );
11.         lista_cores.add ( "vermelho" );
12.         lista_cores.add ( "Laranja" );
13.         lista_cores.add ( "Verde" );
14.         lista_cores.add ( "branco" );
15.         lista_cores.add ( "Amarelo" );
16.         lista_cores.add ( "Preto" );
17.         System.out.println ( lista_cores );
18.     }
19.     @Override
20.     public int compare ( String obj1 , String obj2 )
21.     {
22.         int cmp;
23.         if ( ( cmp = obj1.compareTo ( obj2 ) ) < 0 )
24.             return 1;
25.         else if ( cmp > 0 )
26.             return -1;
27.         else
28.             return 0;
29.     }
30. }
31.
```

Assinale a única opção que exibe a saída correta do código.

A

[Amarelo, azul, Branco, branco, Laranja, Preto, Verde, vermelho].

B

[vermelho, Verde, Preto, Laranja, Branco, azul, Amarelo].

C

[vermelho, Verde, Preto, Laranja, branco, Branco, azul, Amarelo].

D

[Branco, azul, vermelho, Laranja, Verde, branco, Amarelo, Preto].

E

[Amarelo, azul, Branco, Laranja, Preto, Verde, vermelho].

A alternativa D está correta.

ArrayList é uma estrutura que não ordena seus elementos. Assim, a versão especializada do método “compare” fornecida na linha 20 não impacta a inserção dos elementos. Isso significa também que ela não impede elementos duplicados. Logo, ao se imprimir os elementos da estrutura, eles serão impressos na ordem em que foram inseridos.

O método 'toString'

Em Java, todas as classes descendem direta ou indiretamente da classe Object. Essa classe provê um conjunto de métodos que, pela herança, estão disponíveis para as subclasses. Tais métodos oferecem ao programador a facilidade de contar com comportamentos comuns fornecidos pela própria biblioteca da linguagem, além de poderem especializar esses métodos para atenderem às necessidades particulares de sua implementação.

A partir de agora, abordaremos o método “toString” da classe Object e veremos como usá-lo aplicando o conceito de herança.

Entenda neste vídeo a utilidade e as formas de empregar o método “toString” derivado da classe Object em Java.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Esse método permite identificar o objeto retornando uma representação “String” deste. Ou seja, é de uma representação textual que assume a forma:

<nome completamente qualificado da classe à qual o objeto pertence>@<código hash do objeto>.

O <nome completamente qualificado da classe à qual o objeto pertence> é a qualificação completa da classe, incluindo o pacote ao qual ela pertence. A qualificação é seguida do símbolo de “@” e, após este, do código hash do objeto. Veja um exemplo de saída do método “toString” a seguir.

```
java

Com.mycompany.GereEscola.Juridica@72ea2f77
```

Nessa saída, notamos o pacote (com.mycompany.GereEscola), a classe (Juridica) e o hash do objeto (72ea2f77) formando o texto representativo daquela instância.

A implementação do método “toString” é mostrada no Código 28. Nela identificamos o uso de outros métodos da classe Object, como “getClass” e “hashCode”. Não vamos nos aprofundar no primeiro, mas convém dizer que ele retorna um objeto do tipo Class, que é a representação da classe em tempo de execução do objeto. O segundo será visto adiante.

Código 28: Método “toString”:

```
java

public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

Essa é a implementação padrão fornecida pela biblioteca Java, que pode ser prontamente usada pelo programador. Entretanto, prover uma especialização é interessante, pois permite dar um formato mais útil à

saída do método. Nesse caso, os mecanismos de polimorfismo e herança permitirão que o método mais especializado seja empregado. Remetendo à saída anteriormente mostrada, vemos que há pouca informação, prejudicando sua utilidade. Além de saber o nome completamente qualificado da classe, temos somente o seu código hash. Assim, vamos modificar o método “toString” para que ele apresente outras informações, conforme vemos na linha 18 da implementação da classe Pessoa parcialmente mostrada no Código 29. A classe Pessoa tem Fisica como subclasse. A implementação de Fisica é mostrada no Código 30.

Código 29: Classe Pessoa — código parcial com método "toString" especializado:

```
java

public class Pessoa {
    //Atributos
    protected String nome , naturalidade , nacionalidade , identificador;
    private Calendar data_inicio_existencia;
    private int idade;
    private Endereco endereco;

    //Métodos
    public Pessoa ( String nome , Calendar data_inicio_existencia, String identificador ,
Endereco endereco , String nacionalidade , String naturalidade ) {
        this.nome = nome;
        this.data_inicio_existencia = data_inicio_existencia;
        this.identificador = identificador;
        this.endereco = endereco;
        this.nacionalidade = nacionalidade;
        this.naturalidade = naturalidade;
    }
...
public String toString (){
        return "Objeto:" + "\n\t- Classe: " + getClass().getName() + "\n\t- Hash: " +
Integer.toHexString(hashCode()) + "\n\t- Nome: " + nome + "\n\t- Identificador: "
+ identificador;
    }
}
```

Trecho de código da Classe Fisica (derivada de Pessoa).

Código 30: Classe Fisica (derivada de Pessoa):

```
java

public class Fisica extends Pessoa {
    //Atributos

    //Métodos
    public Fisica ( String nome , Calendar data_nascimento , String CPF , Endereco
endereco , String nacionalidade , String naturalidade ) {
        super ( nome , data_nascimento, CPF , endereco , nacionalidade , naturalidade );
        atualizarIdade ();
    }
}
```

O Código 31 invoca o método “toString” para dois objetos criados, um do tipo Pessoa e outro do tipo Fisica.

Código 31:

```
java

public class Principal {
    private static Pessoa grupo [];
    public static void main (String args[]) {
        grupo = new Pessoa [2];
        grupo [0] = new Fisica ( "Marco Antônio" , data_nasc , “365.586.875-45”, null ,
"Brasil" , "Rio de Janeiro" );
        grupo [1] = new Pessoa ("Escola Novo Mundo Ltda" , data_nasc ,
“43.186.666/0026-32” , null , "Brasil" , "Rio de Janeiro");
        for ( int i = 0 ; i <= 1 ; i++ )
        System.out.println( "grupo[" + i + “]: “ + grupo[i].toString() );
    }
```

Confira a saída da execução do Código 31!

```
java

grupo[0]: Objeto:
        - Classe: com.mycompany.GereEscola.Fisica
        - Hash: 77459877
        - Nome: Marco Antônio
        - Identificador: 365.586.875-45
grupo[1]: Objeto:
        - Classe: com.mycompany.GereEscola.Pessoa
        - Hash: 378bf509
        - Nome: Escola Novo Mundo Ltda
        - Identificador: 43.186.666/0026-32
```

Veja que o método “toString” agora apresenta outra composição de saída, trazendo informações como o nome e o identificador. Além disso, formatamos a saída para apresentar as informações em múltiplas linhas identadas.

Se olharmos novamente o Código 31, veremos que na linha 5 o objeto criado é do tipo Fisica. Apesar de Fisica não reescrever o método “toString”, ela herda a versão especializada de sua superclasse, que é invocada no lugar da versão padrão. Olhando também para a saída, percebemos que o objeto foi corretamente identificado como sendo da classe Fisica.

Ao modificar a implementação do método, tornamos seu retorno mais útil. Por esse motivo, ele é frequentemente sobrescrito nas diversas classes providas pela biblioteca Java.

Atividade 1

Sobre o método “toString”, assinale a alternativa correta.

A

É o método invocado quando se deseja obter o código hash de um objeto.

B

É o método invocado quando se deseja obter a classe de um objeto.

C

Converte o objeto em uma String.

D

Fornece uma representação textual do objeto que pode ser modificada pelo programador.

E

Sempre retorna um identificador único do objeto.

A alternativa D está correta.

O método “toString” pertence à classe Object e seu objetivo é fornecer uma descrição textual que represente o objeto. A própria documentação da linguagem recomenda que ele seja sobrescrito nas subclasses para que sempre seja representativo da classe definida.

Os métodos 'equals' e 'hashcode'

Outros dois métodos herdados da classe Object são “equals” e “hashCode”. Da mesma maneira que o método “toString”, eles têm uma implementação provida por Object, mas também são métodos que usualmente iremos especializar a fim de que tenham semântica útil para as classes criadas.

Apesar de dedicarmos este tópico aos métodos citados, não podemos deixar de mencionar o operador “instanceof”, o qual permite que comparemos o objeto com um típico específico. Assim, veremos adiante como utilizar esses dois métodos e o operador “instanceof”.

Neste vídeo, você vai entender a utilidade e o modo de empregar os métodos "equals" e "hashcode" derivados da classe Object em Java. Não perca!

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O método “equals” é utilizado para avaliar se outro objeto é igual ao objeto que invoca o método. Se forem iguais ele retorna “true”; caso contrário, ele retorna “false”. A sua assinatura é “boolean equals (Object obj)” e sua implementação é mostrada no Código 32. Visto que ele recebe como parâmetro uma referência para um objeto da classe Object, da qual todas as classes descendem em Java, ele aceita referência para qualquer objeto das classes derivadas.

Código 32: Método "equals" da classe Object:

```
java

public boolean equals(Object obj) {
    return (this == obj);
}
```

A implementação de “equals” busca ser o mais precisa possível em termos de comparação, retornando verdadeiro (“true”) se, e somente se, os dois objetos comparados são o mesmo objeto. Ou seja, “equals” implementa uma relação de equivalência entre referências não nulas de objetos tal que as seguintes propriedades são válidas. Tome nota!

Reflexividade

Qualquer que seja uma referência não nula R, R.equals(R) retorna sempre “true”.

Simetria

Quaisquer que sejam as referências não nulas R e S, R.equals(S) retorna “true” se, e somente se, S.equals(R) retorna “true”.

Transitividade

Quaisquer que sejam as referências não nulas R, S e T, se R.equals(S) retorna “true” e S.equals(T) retorna “true”, então R.equals(T) deve retornar “true”.

Consistência

Quaisquer que sejam as referências não nulas R e S, múltiplas invocações de R.equals(S) devem consistentemente retornar “true” ou consistentemente retornar “false”, dado que nenhuma informação do objeto usada pela comparação em “equals” tenha mudado; qualquer que seja uma referência não nula R, R.equals(null) retorna sempre “false”.

Usualmente, a implementação de “equals” é sobrescrita na classe derivada. Quando isso ocorre, é dever do programador garantir que essa relação de equivalência seja mantida. Obviamente nada impede que ela seja quebrada, dando uma nova semântica ao método, mas se isso ocorrer, então deve ser o comportamento desejado e não por consequência de erro. Garantir a equivalência pode ser relativamente trivial para tipos de dados simples, mas se torna elaborado quando se trata de classes.

Vamos ver uma aplicação simples de “equals”. Para isso, vamos trabalhar com nove objetos, dos quais três são do tipo “int”, três do topo “String” — ambos tipos primitivos — e três são referências para objetos das classes Pessoa e Fisica, conforme vemos no Código 33.

Código 33: Exemplo de uso de “equals”:

```
java

public class Principal {
    //Atributos
    private static int I1 , I2 , I3;
    private static String S1 , S2 , S3;
    private static Fisica p1 , p2;
    private static Pessoa p3;

    //Métodos
    public static void main (String args[]) {
        I1 = 1;
        I2 = 2;
        I3 = 1;
        S1 = "a";
        S2 = "b";
        S3 = "a";
        Calendar data_nasc = Calendar.getInstance();
        data_nasc.set(1980, 10, 23);
        p1 = new Fisica ( "Marco Antônio" , data_nasc , "365.586.875-45" , null ,
"Brasil" , "Rio de Janeiro" );
        p2 = new Fisica ( "Marco Antônio" , data_nasc , "365.586.875-45" , null ,
"Brasil" , "Rio de Janeiro" );
        p3 = new Pessoa ( "Classe Pessoa" , null , null , null , "Brasil" , "Rio de
Janeiro");
        compararEquals ( p1 , p2 , p3 );
        compararEquals ( I1, I2, I3 );
        compararEquals ( S1, S2, S3 );
    }

    private static void compararEquals ( Object o1 , Object o2 , Object o3 ){
            System.out.println("Uso de EQUALS para comparar " + o1.getClass().getName());
            if ( o1.equals( o2 ) )
                System.out.println("o1 == o2");
            else
                System.out.println("o1 != o2");
            if ( o1.equals(o3) )
                System.out.println("o1 == o3");
            else
                System.out.println("o1 != o3");
        }
    }
```

Nas linhas 21, 22 e 23 do Código 33, o que fazemos é comparar os diversos tipos de objetos utilizando o método “equals”. O resultado é mostrado a seguir.

```
java

Uso de EQUALS para comparar com.mycompany.GereEscola.Fisica
o1 != o2
o1 != o3
Uso de EQUALS para comparar java.lang.Integer
o1 != o2
o1 == o3
Uso de EQUALS para comparar java.lang.String
o1 != o2
o1 == o3
```

Podemos ver que o objeto “I1” foi considerado igual ao “I3”, assim como “S1” e “S3”. Entretanto, “p1” e “p2” foram considerados diferentes, apesar de serem instâncias da mesma classe (Fisica) e seus atributos serem rigorosamente iguais. Por quê?

A resposta a essa pergunta mobiliza vários conceitos, a começar pelo entendimento do que ocorre nas linhas 5 e 6. As variáveis “p1”, “p2” e “p3” são referências para objetos das classes Fisica e Pessoa. Em contrapartida, as variáveis “I1”, “I2”, “I3”, “S1”, “S2” e “S3” são todas tipos primitivos. O operador de comparação “==” atua comparando o conteúdo das variáveis que, para os tipos primitivos, são os valores neles acumulados. Mesmo o tipo “String”, que é uma classe, tem o seu valor acumulado avaliado. O mesmo ocorre para os tipos de dados de usuário (classes), só que nesse caso “p1”, “p2” e “p3” armazenam o endereço de memória (referência) onde os objetos estão. Como esses objetos ocupam diferentes endereços de memória, a comparação entre “p1” e “p2” retorna “false” (são objetos iguais, mas não são o mesmo objeto). Uma comparação “p1.equals(p1)” retornaria “true” obviamente, pois nesse caso trata-se do mesmo objeto.



Comentário

Segundo a documentação da linguagem, a semântica de “equals” na classe Object é retornar verdade se, e somente se, estivermos comparando o mesmo objeto: “this method returns true if and only if x and y refer to the same object (x == y has the value true)” (ORACLE INC., s.d.). Mas diferentemente do operador “==”, o método “equals” realiza a comparação com base na implementação realizada, o que permite dar-lhe outra semântica, isto é, retornar “true” se o os conteúdos dos objetos comparados forem iguais, por exemplo.

Fica claro porque precisamos sobrescrever “equals” se quisermos comparar objetos definidos pelo usuário. No Código 34 mostramos a reimplementação desse método para a classe Pessoa.

Código 34: Método "equals" especializado:

```
java

public boolean equals (Object obj) {
    if ( ( nome.equals( ( ( Pessoa ) obj ).nome ) ) && ( this instanceof Pessoa ) )
        return true;
    else
        return false;
}
```

Com essa especialização, veja a saída!

```
java

Uso de EQUALS para comparar com.mycompany.GereEscola.Fisica
o1 == o2
o1 != o3
```

Contudo, da forma como implementamos “equals”, “p1” e “p2” serão considerados iguais mesmo que os demais atributos sejam diferentes. Esse caso mostra a complexidade que mencionamos em se estabelecer a relação de equivalência entre objetos complexos. Cabe ao programador determinar quais características devem ser consideradas na comparação.

A reimplementação de “equals” impacta indiretamente o método “hash”. A própria documentação de “equals” aponta isso quando diz “que geralmente é necessário substituir o método ‘hashCode’ sempre que esse método (‘equals’) for substituído, de modo a manter o contrato geral para o método ‘hashCode’, que afirma que objetos iguais devem ter códigos hash iguais” (ORACLE AMERICA INC., 2021b).

Isso faz todo sentido, já que um código hash é uma impressão digital de uma entidade. Pense no caso de arquivos. Quando baixamos um arquivo da internet, como uma imagem de um sistema operacional, usualmente calculamos o hash para verificar se houve erro no download. Isto é, mesmo sendo duas cópias distintas, esperamos que se forem iguais os arquivos tenham o mesmo hash. Esse princípio é o mesmo por trás da necessidade de se reimplementar “hashCode”.

Nós já vimos o uso desse método quando estudamos “toString” e a saída do Código 31 mostra o resultado do seu emprego. A sua assinatura é “int hashCode()” e ele tem como retorno o código hash do objeto. Conheça algumas propriedades desse método.

1

Invocações sucessivas sobre o mesmo objeto devem consistentemente retornar o mesmo valor, uma vez que nenhuma informação do objeto usada pela comparação em “equals” tenha mudado.

2

Se dois objetos são iguais segundo “equals”, então a invocação de “hashCode” deve retornar o mesmo valor para ambos.

3

Caso dois objetos sejam desiguais segundo “equals”, não é obrigatório que a invocação de “hashCode” em cada um dos dois objetos produza resultados distintos. Mas produzir resultados distintos para objetos desiguais pode melhorar o desempenho das tabelas hash.

A invocação de “hashCode” nos objetos “p1” e “p2” do Código 33 nos dá a seguinte saída:

```
java

p1 = 2001049719
p2 = 250421012
```

Esse resultado contraria as propriedades de “hashCode”, pois nossa nova implementação de “equals” estabeleceu a igualdade entre “p1” e “p2” instanciados no Código 31. Para restaurar o comportamento correto, fornecemos a especialização de “hashCode” vista no Código 35.

Código 35: Reimplementação de "hashCode":

```
java

public int hashCode () {
    if ( this instanceof Pessoa )
        return this.nome.hashCode();
    else
        return super.hashCode();
}
```

Veja que agora nossa implementação está consistente com a de “equals”. Na linha 2 do Código 35, verificamos que se trata de um objeto da classe Pessoa. Se não for, chamamos a implementação padrão de “hashCode”. Caso seja, retornamos o valor de hash da “String” que forma o atributo “nome”, o mesmo usado em “equals” para comparação.

Confira a saída!

```
java

p1 = -456782095
p2 = -456782095
```

Agora, conforme esperado, “p1” e “p2” possuem os mesmos valores.

Em ambos os casos, oferecemos uma reimplementação simples para corrigir o comportamento, mas você deve estar preparado para necessidades bem mais complexas. O programador precisa, no fundo, compreender o comportamento modelado e as propriedades desses métodos, para que sua modificação seja consistente.

Operador “Instanceof”

Esse operador é utilizado para comparar um objeto com um tipo específico. É o único operador de comparação de tipo fornecido pela linguagem. Sua sintaxe é simples: “op1 instanceof op2”, onde o operando “op2” é o tipo com o qual se deseja comparar e “op1” é o objeto ou expressão a ser comparada.

Um exemplo do uso desse operador é mostrado na linha 2 do Código 35. O operador retorna verdadeiro se “op1” for uma instância do tipo “op2”. Então devemos esperar que se “op1” for uma subclasse de “op2”, o retorno será “true”. Vejamos se isso é válido executando o Código 36.

Código 36: Operador "instanceof":

```
java

public class Principal {
    //Atributos
    private static Pessoa p1, p3;
    private static Fisica p2;
    //Métodos
    public static void main (String args[]) {
        Calendar data_nasc = Calendar.getInstance();
        data_nasc.set(1980, 10, 23);
        p1 = new Fisica ( "Marco Antônio" , data_nasc , "365.586.875-45" , null ,
"Brasil" , "Rio de Janeiro" );
        p2 = new Fisica ( "Marco Antônio" , data_nasc , "365.586.875-45" , null ,
"Brasil" , "Rio de Janeiro" );
        p3 = new Pessoa ( "Classe Pessoa" , null , null , null , "Brasil" , "Rio de
Janeiro");
    if ( p1 instanceof Pessoa )
       System.out.println("p1 é instância do tipo Pessoa.");
    else
       System.out.println("p1 não é instância do tipo Pessoa.");
    if ( p2 instanceof Pessoa )
       System.out.println("p2 é instância do tipo Pessoa.");
    else
       System.out.println("p2 não é instância do tipo Pessoa.");
    if ( p3 instanceof Pessoa )
       System.out.println("p3 é instância do tipo Pessoa.");
    else
       System.out.println("p3 não é instância do tipo Pessoa.");
    if ( p3 instanceof Fisica )
      System.out.println("p3 é instância do tipo Fisica.");
    else
         System.out.println("p3 não é instância do tipo Fisica.");
    }
}
```

Nesse código fazemos 4 comparações. Veja os resultados!

```
java

p1 é instância do tipo Pessoa.
p2 é instância do tipo Pessoa.
p3 é instância do tipo Pessoa.
p3 não é instância do tipo Fisica.
```

Assim, o código valida nossa expectativa: “instanceof” retorna verdadeiro para uma instância da subclasse quando comparada ao tipo da superclasse. Tal resultado independe se a variável que a referencia é uma referência para a superclasse (linha 9) ou a própria subclasse (linha 10).

Atividade 2

Sobre os métodos “equals” e “hashCode” da classe Objects, assinale a alternativa correta.

A

Mudanças em “equals” podem ensejar mudanças em “hashCode”, mas o inverso nunca ocorre.

B

Como “equals” é herdado por todos os objetos, não necessita ser modificado para estabelecer a igualdade de objetos.

C

O método “equals” sempre depende de “hashCode” para estabelecer a igualdade.

D

A modificação de um sempre imporá a modificação de outro.

E

Objetos iguais terem código hash distintos viola o contrato geral de “hashCode”.

A alternativa E está correta.

O contrato geral do método “hashCode” afirma que objetos iguais devem ter códigos hash iguais.

Entendendo o acesso protegido

De maneira geral, o acesso protegido restringe o acesso aos atributos de uma classe ao pacote dessa classe e às suas subclasses. Portanto, uma subclasse acessará todos os métodos e atributos públicos ou protegidos da superclasse, mesmo se pertencerem a pacotes distintos. Claro que se forem pacotes diferentes, a subclasse deverá importar primeiro a superclasse.

Agora vamos nos deter em algumas situações particulares.

Assista ao vídeo e confira mais detalhadamente o conceito de acesso protegido e como a atribuição de acesso protegido afeta a visibilidade de variáveis e métodos.

> Conteúdo interativo
>
> Acesse a versão digital para assistir ao vídeo.

Para nosso estudo, vamos criar o pacote Matematica e, a partir de agora, as classes que temos utilizado (Principal, Pessoa, Fisica, Juridica, Aluno etc.) estarão inseridas no pacote GereEscola. No pacote Matematica vamos criar a classe Nota, que implementa alguns métodos de cálculos relativos às notas dos alunos. Essa classe pode ser parcialmente vista no Código 37.

Código 37: Código parcial da classe Nota:

```
java

package com.mycompany.Matematica;
...
public class Nota {
...
    public float calcularMedia () {
...
    }
    protected float calcularCoeficienteRendimento () {
...
    }
...
}
```

No pacote GereEscola, vamos criar a classe Desempenho, conforme mostrado parcialmente em Código 38.

Código 38: Classe Desempenho:

```
java

//Pacote
package com.mycompany.GereEscola;

//Importações
import com.mycompany.Matematica.Nota;

//Classe
public class Desempenho extends Nota {
    //Atributos
    private float media , CR;
    private Nota nota;

    //Métodos
    public Desempenho () {
        nota = new Nota ();
        media = calcularMedia();
        CR = calcularCoeficienteRendimento();
        //media = nota.calculaMedia();
        //CR = nota.caulculaCoeficienteRendimento();
    }
}
```

A classe Desempenho é filha da classe Nota. Por isso, ela tem acesso aos métodos “calcularMedia” (público) e “calcularCoeficienteRendimento” (protegido) de Nota, mesmo estando em outro pacote.

Observando a linha 11, vemos que ela também possui um objeto do tipo Nota, instanciado na linha 15. Sendo assim, será que poderíamos comentar as linhas 15 e 16, substituindo-as pelas linhas 18 e 19? A resposta é não. Se descomentarmos a linha 18, não haverá problema, mas a linha 19 causará erro de compilação. Isso ocorre porque Desempenho tem acesso ao método protegido “calcularCoeficienteRendimento” através da herança. Por esse motivo, ele pode ser invocado diretamente na classe (linha 17). Mas a invocação feita na linha 19 se dá através de uma mensagem enviada para o objeto “nota”, violando a restrição de acesso imposta por protected para objetos de pacotes distintos.

Outra violação de acesso ocorrerá se instanciarmos a classe Desempenho em outra parte do pacote GereEscola e tentarmos invocar o método “calcularCoeficienteRendimento”, como no trecho do Código 39.

Código 39: Exemplo de invocação ilegal de “calcularCoeficienteRendimento”:

```
java

//Pacote
package com.mycompany.GereEscola;
...
public class NovaClasse {
    private static Desempenho des;
...
    public static void main (String args[]) {
        des = new Desempenho ();
        CR = des.calcularCoeficienteRendimento();
...
    }
...
}
```

Na linha 9, a tentativa de invocar “calcularCoeficienteRendimento” vai gerar erro de compilação, uma vez que, embora a classe Desempenho seja do mesmo pacote que NovaClasse, o método foi recebido por Desempenho através de herança de superclasse de outro pacote. Logo, como impõe a restrição de acesso protegida, ele não é accessível por classe de outro pacote que não seja uma descendente da superclasse.

Atividade 3

Julgue estas afirmações:

I. Métodos protegidos de uma subclasse são diretamente acessíveis para a superclasse quando no contexto desta.

II. Os atributos e métodos protegidos são herdados e acessíveis pela subclasse.

III. Métodos protegidos são acessíveis por classes fora da hierarquia desde que estejam no mesmo pacote.

Está correto o que se afirma em:

A

I.

B

II.

C

III.

D

I e II.

E

II e III.

A alternativa B está correta.

O modificador de acessibilidade protected restringe o acesso a um atributo ou método apenas às classes da hierarquia. Contudo, a superclasse não possui acesso aos métodos e atributos da subclasse pela simples razão de que ela não os herda. A hierarquia funciona legando métodos e atributos das classes genéricas para as especializadas, e não ao inverso.

Os principais métodos da classe Object na prática

Mediante um exemplo simples, vamos explorar os conceitos que vimos anteriormente. Para isso, usaremos as classes Principal, PessoaFisica e Aluno com algumas alterações.

A classe Principal, que implementa a interatividade, cria algumas situações de comparação entre objetos. Isso nos permitirá estudar a igualdade de objetos de Java através do operador "==" e do método "equals". A classe PessoaFisica traz como novidade os métodos "equals", "hashCode" e "toString" sobrescritos. A classe Aluno estende a classe PessoaFisica e também sobrescreve esses métodos. Como Aluno é um subtipo de PessoaFisica, ele herda os métodos e atributos desta. Nesse caso, convém notar que, em PessoaFisica, os atributos são protegidos (protected) e o método "validarCPF" é privado.

Com esse exemplo prático, podemos explorar o que está envolvido na reescrita dos métodos "equals ()" e "toString" em tipos definidos por usuários e como o acesso protegido impacta a herança.

Como sempre, estimulamos você a alterar o código apresentado e explorar outros aspectos. O assunto é amplo, e estas poucas linhas nunca seriam capazes de esgotá-lo, por isso recomendamos aprofundar seus estudos. Mas se ligue nesta dica: quando estiver estudando ou tentando aprender algo, você pode se valer do que outros escreveram sobre aquilo, mas procure sempre a fonte. Às vezes, detalhes se perdem e conhecê-los é um dos diferenciais entre o bom profissional e o especialista.

Assista ao vídeo e conheça os principais usos práticos dos métodos da classe Object em Java.

Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Roteiro de prática

Vamos começar com o código da classe PessoaFisica (Código 40)!

Código 40: Classe PessoaFisica:

```
java
```

```
public class PessoaFisica {
    protected String CPF;
    protected String nome;

    public PessoaFisica ( String nome , String CPF ) {
        this.nome = nome;
        setCPF ( CPF );
    }
    public void setNome ( String nome ) {
        this.nome = nome;
    }
    public String getNome ( ) {
        return nome;
    }
    public void setCPF ( String CPF ) {
        if ( validarCPF ( CPF ) )
            this.CPF = CPF;
        else
            System.out.println ( "CPF invalido!" );
    }
    public String getCPF ( ) {
        return CPF;
    }
    @Override
    public boolean equals ( Object obj ) {
        if ( this.CPF == ( ( PessoaFisica ) obj ).CPF )
            return true;
        else
            return false;
    }
    @Override
    public int hashCode ( ) {
        return CPF.hashCode();
    }
    @Override
    public String toString () {
        return String.format ( "Classe: <%s> | Hash: %s" , "PessoaFisica" ,
Integer.toHexString ( this.hashCode() ).toUpperCase() );
    }
    private boolean validarCPF ( String CPF ) {


        //verifica sequência de dígitos iguais e tamanho (11 dígitos)
        if ( ! ( CPF.chars ( ).allMatch ( Character::isDigit ) ) ) {
            System.out.println ( "CPF possui caracteres não numéricos!" );
            return false;
        }
        else if ( ( CPF.length() != 11 ) || CPF.matches( "[0]{11}|[1]{11}|[2]{11}|[3]{11}|
[4]{11}|[5]{11}|[6]{11}|[7]{11}|[8]{11}|[9]{11}" ) )
            return false;
        return true;
    }
}
```

Veja que a semântica do método “equals” é que dois objetos dessa classe são iguais se, e somente se, os atributos CPF são iguais. Isso quer dizer que dois objetos com atributo “nome” iguais, mas “CPF” diferentes serão considerados objetos distintos. Em um caso real, essa lógica de implementação irá derivar diretamente da análise.

Vamos fazer um pequeno desvio para aproveitar a oportunidade e esclarecer brevemente a importância dos diversos papéis em um desenvolvimento. A regra acima pode parecer óbvia, sendo de imediata apreensão pelo programador, mas nosso exemplo tem menos de 300 linhas e 3 classes. Em situações reais, sistemas serão bem mais complexos com pelo menos dezenas de tipos definidos pelo programador. Assim, garantir que todos modelem corretamente as regras de negócio e prevejam os testes adequados exige o trabalho de um especialista. Esse é o escopo do trabalho do analista de software. Muitas vezes, o programador assume todos os papéis levando, não sem surpresa, a um produto com vícios de desenvolvimento (defeitos).

Voltando ao nosso exemplo, uma vez que definimos a semântica de “equals”, precisamos ajustar o método “hashCode”. Como dissemos, é preciso manter o contrato geral do método de forma que dois objetos considerados iguais tenham o mesmo retorno de “hashCode”. Vemos na linha 32 que isso foi obtido fazendo com que o método retorne o código hash do atributo CPF.

Agora vamos ver a classe Aluno (Código 41)!

Código 41: Classe Aluno:

```
java

public class Aluno extends PessoaFisica {
    String matricula;

    public Aluno ( String nome , String CPF , String matricula ) {
        super ( nome , CPF );
        this.matricula = matricula;
    }
    public void setMatricula ( String matricula ) {
        this.matricula = matricula;
    }
    public String getMatricula ( ) {
        return matricula;
    }
    @Override
    public boolean equals ( Object obj ) {
        if ( ( this.CPF == ( ( Aluno ) obj ).CPF ) && ( this.matricula == ( ( Aluno )
obj ).matricula ) )
            return true;
        else
            return false;
    }
    @Override
    public int hashCode ( ) {
        return ( this.CPF + this.matricula ).hashCode();
    }
    @Override
    public String toString () {
        return String.format ( "Classe: <%s> | Hash: %s" , "Aluno" , Integer.toHexString
( hashCode() ).toUpperCase() );
    }
}
```

Em Aluno, passamos a considerar que dois objetos são iguais se, e somente se, possuem os mesmos atributos “CPF” e “matricula” (linha 15). Assim, ajustamos o método “hashCode” para refletir isso. Como se vê na linha 22, agora “hashCode” retorna o código hash da concatenação dos atributos “CPF” e “matricula”.

Em ambas as classes, nós reescrevemos o método “toString” incluindo, entre outras modificações, o tipo do objeto.

Vamos agora examinar a saída da execução da classe Principal.

Código 42: Classe Principal:

```
java

public class Principal {
    public static void main ( String args [ ] ) {
        PessoaFisica PF1 = new PessoaFisica ( "Ricardo" , "12345678998" );
        PessoaFisica PF2 = new PessoaFisica ( "Ricardo" , "98774112336" );
        PessoaFisica PF3 = new PessoaFisica ( "Carlos" , "98774112336" );
        Aluno A1 = new Aluno ( "Ricardo" , "12345678998" , "AL001" );
        Aluno A2 = new Aluno ( "Ricardo" , "12345678998" , "AL001" );
        Aluno A3 = new Aluno ( "Carlos" , "12345678998" , "AL002" );
        //PF1 = PF2 ?
        System.out.println ( "Comparando PF1 e PF2" );
        System.out.println ( "  PF1.equals ( PF2 ) -> " + PF1.equals ( PF2 ) );
        //PF1 = PF3 ?
        System.out.println ( "Comparando PF1 e PF3" );
        System.out.println ( "  PF1.equals ( PF3 ) -> " + PF1.equals ( PF3 ) );
        //PF2 = PF3 ?
        System.out.println ( "Comparando PF2 e PF3" );
        System.out.println ( "  PF2.equals ( PF3 ) -> " + PF2.equals ( PF3 ) );
        System.out.println ( "  PF2 == PF3 -> " + ( PF2 == PF3 ) );
        //A1 = A2 ?
        System.out.println ( "Comparando A1 e A2" );
        System.out.println ( "  A1.equals ( A2 ) -> " + A1.equals ( A2 ) );
        System.out.println ( "  A1 == A2 -> " + ( A1 == A2 ) );
        //A1 = A3 ?
        System.out.println ( "Comparando A1 e A3" );
        System.out.println ( "  A1.equals ( A3 ) -> " + A1.equals ( A3 ) );
        //PF2 = PF3 ?
        System.out.println ( "Comparando A2 e A3" );
        System.out.println ( "  A2.equals ( A3 ) -> " + A2.equals ( A3 ) );
        //PF1 = A1 ?
        System.out.println ( "Comparando PF1 e A1" );
        System.out.println ( "  PF1.equals ( A1 ) -> " + PF1.equals ( A1 ) );
        System.out.println ( "  PF1 == A1 -> " + ( PF1 == A1 ) );
        //A1 = PF1 ?
        System.out.println ( "Comparando PF1 e A1" );
        System.out.println ( "    PF1.equals ( A1 ) -> " + PF1.equals ( A1 ) );
        //Imprime o identificador do objeto
        System.out.println ( "[PF1]: "+PF1 + " [PF2]: " + PF2 + " [PF3]: " + PF3 );
        System.out.println ( "[A1]: " + A1 + " [A2]: " + A2 + " [A3]: " + A3 );
    }
}
```

A saída da execução do código pode ser vista a seguir.

```
java

--------------------------------[ jar ]---------------------------------
--- exec-maven-plugin:3.1.0:exec (default-cli) @ Tema2_Modulo_2 ---
Comparando PF1 e PF2
  PF1.equals ( PF2 ) -> false
Comparando PF1 e PF3
  PF1.equals ( PF3 ) -> false
Comparando PF2 e PF3
  PF2.equals ( PF3 ) -> true
  PF2 == PF3 -> false
Comparando A1 e A2
  A1.equals ( A2 ) -> true
  A1 == A2 -> false
Comparando A1 e A3
  A1.equals ( A3 ) -> false
Comparando A2 e A3
  A2.equals ( A3 ) -> false
Comparando PF1 e A1
  PF1.equals ( A1 ) -> true
  PF1 == A1 -> false
Comparando PF1 e A1
    PF1.equals ( A1 ) -> true
[PF1]: Classe:  | Hash: 2C6EDE14 [PF2]: Classe:  | Hash: D83C8899 [PF3]: Classe:  | Hash:
D83C8899
[A1]: Classe:  | Hash: 62040C92 [A2]: Classe:  | Hash: 62040C92 [A3]: Classe:  | Hash:
62040C93
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Observe a linha 35. Nela comparamos um objeto do tipo PessoaFisica com outro do tipo Aluno e o resultado foi “true”. O que acontece é que ao invocarmos “equals”, estamos fazendo-o através de um objeto PessoaFisica e assim comparamos apenas o atributo “CPF”. Como Aluno é subtipo de PessoaFisica e “CPF” é um atributo protegido, a comparação pode ser feita. No caso avaliado, ambos os objetos possuem o mesmo valor de “CPF”.

Agora que compreendemos o funcionamento do programa, faça alterações no código e veja o resultado. Por exemplo, o que aconteceria se fizéssemos “A1.equals (PF1);”? Não se contente apenas com o resultado, entenda o porquê daquele resultado, mesmo se a consequência for um erro.

Faça você mesmo!

Um programador define a classe Pessoa e a classe Fisica que a estende. Parte do código de Pessoa é exibido a seguir, junto de parte do código de Fisica. Sabe-se que Fisica não sobrescreve o método “equals”.

```java
1. public class Pessoa {
2. //Atributos
3.     protected String nome;
4.     protected String identificador;
5.
6.     public Pessoa ( String nome , String identificador ) {
7.     this.nome = nome;
8.     this.identificador = identificador;
9.     }
10. //Métodos
11.     @Override
12.     public boolean equals ( Object obj ) {
13.         if ( this.identificador == ( ( Pessoa ) obj ).identificador )
14.         return true;
15.         else
16.         return false;
17.     }
18.     @Override
19.     public int hashCode () {
20.         return identificador.hashCode();
21.     }
22. }
```

```java
1. public class Fisica extends Pessoa {
2. //Métodos
3.     public Fisica ( String nome , String CPF ) {
4.         super ( nome , CPF );
5.     }
6. }
```

Considere esse trecho de código.

```java
1. Pessoa P1 = new Pessoa ( "Paulo" , "2345" );
2. Fisica A1 = new Fisica ( "Carlos" , "2345" );
3. System.out.println ( P1.equals(A1) );
4. System.out.println ( A1.equals(P1) );
```

A

A saída será "true" e "true".

B

A saída será "true" e "false".

C

A saída será "false" e "true".

D

Haverá erro de compliação causado pela linha 4.

E

Haverá erro de compilação causado pela linha 3.

A alternativa A está correta.

O método “equals” da classe Pessoa não é sobrescrito por Fisica, assim ela o herda como está em Pessoa. Esse método estabelece a comparação de objetos com base no atributo “identificador”. Como Fisica é subtipo de Pessoa e passa, no seu construtor, os valores para a superclasse, a comparação é possível e resulta e true e true.

Considerações finais

O que você aprendeu neste conteúdo?

1. Herança e instanciação de objetos em Java.

2. Hierarquia de herança em Java.

3. Herança, subtipos e princípio da substituição de Liskov.

4. Hierarquia de coleção (Java Collections Framework).

5. Tipos estáticos e dinâmicos.

6. Os métodos "toString", "equals" e "hashcode".

7. Acesso protegido e principais métodos da classe Object.

Explore +

Os princípios SOLID formam um importante conjunto que todo bom programador deve dominar. Neste material vimos apenas um, o princípio da substituição. Sugerimos que você procure e domine também os demais. Para isso, uma busca como os termos princípios SOLID ou com cada um deles deve lhe trazer bons resultados.

Referências

LISKOV, B. H; WING, J. M. **A behavioral notion of subtyping**. ACM Trans. Program. Lang. Syst., v. 16, n. 6, p. 1811–1841, nov. 1994.

ORACLE AMERICA INC. **Lesson: introduction to collections (The JavaTM Tutorials > Collections)**. Oracle Java Documentation, s. d. Consultado na internet em: 22 jan 2021a.

ORACLE AMERICA INC. **Object (Java SE 15 & JDK 15)**. Oracle docs, S. d. Consultado na internet em: 20 fev 2021b.

ORACLE AMERICA INC. **Primitive data types (The JavaTM Tutorials > Learning the Java Language > Language Basics)**. Oracle Java Documentation, s. d. Consultado na internet em: 17 fev 2021c.

ORACLE INC. **Object (Java SE 17 & JDK 17)**. Oracle docs, s. d. Consultado na internet em: 13 maio 2023a.

ORACLE INC. **TreeSet (Java Platform SE 8)**. Oracle docs, s. d. Consultado na internet em: 13 maio 2023b.

TIOBE. **TIOBE Index**. S. d. Consultado na internet em: 6 maio 2023.

URMA, R.-G.; WARBURTON, R. **Java 10 local variable type inference**. Oracle, 10 maio 2022. Consultado na internet em: 17 fev 2021.

VANSHIKA4042. **Top 10 programming languages to learn in 2023**. GeeksforGeeks, s. d. Consultado na internet em: 6 maio 2023